

Article

Blockchain-Enabled Lattice-Based Attribute-Based Searchable Encryption with Instant Revocation

Zhishan Feng, Wenzhong Yang , Ying Hu , Yabo Yin, Tianqi Ma , Xiaodan Tian  and Xiangxin Deng

School of Computer Science and Technology (School of Cyberspace Security), Xinjiang University, Urumqi 830046, China; 107552304052@stu.xju.edu.cn (Z.F.); yinyabo@xju.edu.cn (Y.Y.); 107552304112@stu.xju.edu.cn (T.M.); lianlian@stu.xju.edu.cn (X.T.); yian_deng@stu.xju.edu.cn (X.D.)

* Correspondence: yangwenzhong@xju.edu.cn (W.Y.); huying@xju.edu.cn (Y.H.)

Abstract

As cloud computing proliferates, outsourced data faces severe security threats, yet existing searchable encryption (SE) schemes rely on classical hardness assumptions, centralized trust authorities, and static access control, leaving critical gaps in quantum resistance, single-point-of-failure prevention, and dynamic permission management. To address these limitations, we propose BL-ABSE, a blockchain-enhanced, lattice-based attribute-based searchable encryption framework. BL-ABSE employs the Ring Learning With Errors (RLWE) problem as its security foundation and applies the Number Theoretic Transform (NTT) to reduce polynomial multiplication from $O(n^2)$ to $O(n \log n)$. To eliminate single-point trust risks, the framework further integrates a (t, n) threshold key protocol across an edge-node consortium governed by Practical Byzantine Fault Tolerance (PBFT) consensus. A smart-contract-maintained on-chain revocation list enables permission withdrawal via a single blockchain transaction without re-encryption. Experimental evaluation demonstrates that commitment generation requires approximately 23 ms at $n = 1024$, search latency scales linearly at roughly 29 μ s per record, and revocation completes in approximately 2 s regardless of system scale. Formal security proofs under the quantum polynomial-time (QPT) adversary model reduce six security properties—index indistinguishability, query privacy, threshold key security, Byzantine fault tolerance, audit immutability, and revocation immediacy—to the hardness of RLWE and the Short Integer Solution (SIS) problems. To the best of our knowledge, BL-ABSE is the first framework to simultaneously achieve post-quantum security, attribute-based access control, decentralized key management, instant revocation, and immutable auditing within a single unified framework. We further conduct threshold parameter verification, end-to-end revocation latency decomposition, blockchain throughput stress testing, search-pattern leakage quantification, and communication/storage overhead analysis, providing a comprehensive evaluation of both performance and security trade-offs. We explicitly characterize the search-pattern leakage inherent in the deterministic commitment design as a correctness–privacy trade-off and discuss mitigation directions.



Academic Editor: Shiho Kim

Received: 2 May 2026

Revised: 23 May 2026

Accepted: 27 May 2026

Published: 4 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: searchable encryption; lattice cryptography; ring learning with errors; attribute-based encryption; blockchain; threshold secret sharing

1. Introduction

With the continuous evolution of cloud computing technology, outsourced data storage has become the dominant paradigm for enterprises and individuals managing massive

volumes of information. According to IBM's Cost of a Data Breach Report 2024, the global average cost of a data breach has climbed to an all-time high of \$4.88 million, with approximately 72% of breach incidents involving cloud-hosted data [1]. Highly sensitive sectors such as healthcare, financial services, and e-government administration impose the most stringent requirements on data confidentiality and regulatory compliance, yet cloud service providers may expose sensitive data through insider malicious behavior, misconfiguration, or advanced persistent threat (APT) attacks [2]. How to fully leverage cloud computing resources while preserving data confidentiality has become a core challenge in the field of information security.

Traditional encrypted storage strategies, while effective in preventing plaintext leakage, simultaneously forfeit efficient cloud-side retrieval capabilities—users who wish to search their data must download all ciphertexts, decrypt them locally, and then search, an approach whose time and bandwidth costs are unacceptable at scale [3]. Searchable Encryption (SE) mitigates this limitation by allowing users to perform keyword searches over encrypted data without first decrypting it, thereby striking a balance between confidentiality and usability [4]. Since Song et al. [5] proposed the first symmetric searchable encryption scheme in 2000, the field has developed multiple technical lines including Symmetric Searchable Encryption (SSE) [3], Public-Key Encryption with Keyword Search (PEKS) [6], and Attribute-Based Searchable Encryption (ABSE) [7]. Among these, ABSE combines fine-grained access control with ciphertext retrieval [8], making it especially suitable for multi-user collaborative data-sharing scenarios [7,9].

However, existing attribute-based searchable encryption constructions face three systemic limitations in practical deployment. First, the vast majority of these constructions base their security on the discrete logarithm problem or the bilinear Diffie–Hellman assumption [10,11]. Shor's algorithm, however, has demonstrated that a quantum computer can break these cryptographic primitives in polynomial time [12]. In August 2024, the National Institute of Standards and Technology (NIST) formally published the first post-quantum cryptography standards FIPS 203/204/205 [13–15], signaling that the urgency of cryptographic migration has shifted from academic discussion to engineering deployment [16]. Second, nearly all existing approaches depend on a single trusted authority for key generation and search operations [17]. This centralized trust model is susceptible to single points of failure and key escrow risks, rendering it inadequate for the high-assurance requirements of finance, healthcare, and similar sectors [18]. Third, current permission revocation mechanisms rely heavily on key redistribution or data re-encryption [19], typically incurring latencies of hours or even days and thereby creating serious security windows of vulnerability.

In response to the post-quantum security requirement, prior research has attempted to incorporate lattice-based cryptography into searchable encryption. The FS-PEKS protocol proposed by Zhang et al. [20], a lattice-based forward-secure public-key searchable encryption construction, achieves post-quantum security but incurs $O(n^2)$ polynomial arithmetic overhead far exceeding that of traditional cryptographic approaches. Liu et al. [21] built a post-quantum-secure public-key authenticated searchable encryption construction, but it does not support multi-user access control. Shen et al. [22] designed a lattice-based multi-authority CP-ABE searchable encryption protocol that employs the KUNodes algorithm for attribute revocation; however, revocation complexity is $O(r \cdot \log N)$ and the protocol lacks a blockchain-enhanced trust model. Yang et al. [23] partially introduced NTT optimization, but only for the encryption phase and without support for dynamic permission management.

Regarding the trust model, existing approaches generally assume a single trusted key generation center. The RABAEKS protocol proposed by Cao et al. [24] and the lattice-

based revocable ABE construction by Huang et al. [25] both rely on centralized authorities, resulting in an excessive concentration of privilege. Even ostensibly trustworthy service providers may introduce security vulnerabilities through insider fraud, misconfiguration, or APT attacks [2]. In recent years, several studies have integrated the blockchain with searchable encryption to enhance system trustworthiness [26,27], but these constructions are mostly based on classical cryptographic assumptions and cannot resist quantum attacks.

In terms of dynamic permission management, the attribute revocation mechanism designed by Wang et al. [28] requires key updates for all non-revoked users, with complexity proportional to the number of users. The blockchain-assisted approach of Yu et al. [26] introduces an on-chain revocation mechanism but still necessitates data re-encryption. Research by Chen et al. [29] demonstrates that the time overhead of both major revocation strategies—key update and data re-encryption—scales proportionally with system size, rendering them fundamentally infeasible for large-scale deployments. Moreover, existing solutions provide virtually no operational audit capabilities [30], making it difficult to satisfy the data access audit compliance requirements of regulations such as HIPAA and SOX [30,31].

In response to these challenges, this paper proposes BL-ABSE (Blockchain-Lattice-based Attribute-Based Searchable Encryption), a blockchain-enhanced lattice-based attribute-based searchable encryption framework. BL-ABSE organically integrates the post-quantum security of lattice-based cryptography, the fine-grained access control of attribute-based encryption, and the decentralized trust and tamper-proof auditing of the blockchain. The main contributions are as follows:

(1) Post-quantum-secure and efficient searchable encryption construction. A keyword commitment mechanism based on the RLWE problem is proposed, utilizing the Number Theoretic Transform (NTT) to reduce polynomial multiplication complexity from $O(n^2)$ to $O(n \log n)$. At the parameter $n = 1024$, a 128-bit post-quantum security level is achieved, with commitment generation approximately 17× faster than a naive implementation. Security is strictly reduced to the RLWE decision problem and the Short Integer Solution (SIS) problem.

(2) Decentralized threshold key management mechanism. A (t, n) threshold key protocol is designed that distributes the master key among a federation of edge nodes. User private key generation requires the collaboration of at least t nodes, verified via zero-knowledge proofs, and relies on PBFT consensus to tolerate up to $\lfloor (n - 1)/3 \rfloor$ Byzantine nodes, thereby eliminating single-point trust risks.

(3) Smart-contract-based instantaneous permission revocation. Blockchain smart contracts are used to maintain an on-chain revocation list. Revocation requires only a single blockchain transaction (confirmed in approximately 2 s); all subsequent search requests by a revoked user are immediately rejected without any data re-encryption or key redistribution, effectively reducing revocation latency from the hour scale to the second scale.

(4) Tamper-proof operational audit log. Critical operations including data upload, key request, search query, policy change, and permission revocation are recorded on the blockchain. Cryptographic hash chaining and distributed consensus guarantee audit immutability, facilitating compliance with requirements of regulations such as HIPAA and SOX.

(5) Rigorous formal security analysis. Under the QPT adversary model, sequential game reduction techniques are employed to formally prove that the framework satisfies six security properties: index indistinguishability, query privacy, threshold key security, Byzantine fault tolerance, audit tamper-resistance, and instantaneous revocability. All reductions are founded on the hardness of the RLWE and SIS problems.

Experimental evaluation shows that at $n = 1024$, commitment generation takes approximately 23 ms, search latency is approximately 29 μ s per record, and revocation latency is approximately 2 s. Compared with 13 representative approaches, to the best of our knowledge, BL-ABSE is the first searchable encryption framework to simultaneously achieve post-quantum security, attribute-based access control, decentralized key management, instantaneous revocation, and tamper-proof auditing within a single unified framework.

The remainder of this paper is organized as follows: Section 2 introduces preliminary knowledge on lattice-based cryptography, the Number Theoretic Transform, attribute-based encryption, and blockchain and smart contracts; Section 3 formalizes the problem by presenting the system model, threat model, design objectives, and formal protocol definition; Section 4 describes the protocol design in detail, including each core algorithm and smart contract construction; Section 5 conducts rigorous formal security analysis; Section 6 validates the performance of the framework through a prototype system; Section 7 concludes the paper and discusses future research directions.

2. Preliminaries

This chapter introduces the cryptographic and distributed systems foundations underlying the BL-ABSE framework, establishing the theoretical basis for the subsequent protocol design and security analysis.

2.1. Notation

This paper adopts the following notational conventions: scalars are denoted by lowercase letters (e.g., a , b), vectors by bold lowercase letters (e.g., $\mathbf{a}$, $\mathbf{b}$), and matrices by uppercase letters (e.g., A , B). Table 1 lists the principal symbols and their meanings.

Table 1. Principal symbol descriptions.

Symbol	Description
λ	Security parameter; set to $\lambda = 128$ in this paper
n	Lattice dimension, a power of 2; set to $n = 1024$ in this paper
q	Prime modulus satisfying $q \equiv 1 \pmod{2n}$; set to $q = 12,289$ in this paper
σ	Standard deviation of the discrete Gaussian distribution; set to $\sigma = 3.2$ in this paper
R_q	Polynomial ring, $R_q = \mathbb{Z}_q[x]/(x^n + 1)$
(t, n)	Threshold parameters: t is the threshold value, n is the total number of nodes
H, H_{seed}	Hash function for keyword encoding and hash function for seed derivation

2.2. Foundations of Lattice-Based Cryptography

Lattice-based cryptography is an important branch of post-quantum cryptography whose security is grounded in the computational complexity of hard problems on lattices. No quantum algorithm capable of solving these hard lattice problems in polynomial time has been discovered, making lattices an ideal foundation for constructing quantum-resistant cryptographic systems.

2.2.1. Lattice Definition and Hard Problems

Definition 1 (Lattice). Let $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m \in \mathbb{R}^n$ be m linearly independent vectors. The lattice spanned by these vectors is defined as

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^m z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}, \quad (1)$$

where $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_m]$ is called the basis matrix of the lattice.

Two fundamental hard problems exist on lattices: the Shortest Vector Problem (SVP) requires finding the shortest non-zero vector in the lattice, while the Closest Vector Problem (CVP) requires finding the lattice vector closest to a given target vector. Both problems are NP-hard, and the best known algorithms under the quantum computational model still require exponential time complexity.

2.2.2. LWE and RLWE Problems

Definition 2 (LWE Problem). Given parameters (n, m, q, χ) , the Learning With Errors (LWE) problem asks to distinguish between two distributions: (1) $(\mathbf{A}, \mathbf{b})$, where $\mathbf{A}$ and $\mathbf{b}$ are sampled uniformly at random; and (2) $(\mathbf{A}, \mathbf{As} + \mathbf{e})$, where $\mathbf{s}$ is a secret vector and $\mathbf{e} \leftarrow \chi^m$ is an error vector.

Definition 3 (RLWE Problem). Let the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. The Ring Learning With Errors (RLWE) problem asks to distinguish (a, b) from $(a, a \cdot s + e)$, where $a \in R_q$ is uniformly random and s, e are sampled from an error distribution. Compared with LWE, RLWE compresses the key size from $O(n^2)$ to $O(n)$ and supports $O(n \log n)$ fast polynomial multiplication via NTT.

Assumption 1 (RLWE Assumption). For appropriately chosen parameters, the advantage of any PPT algorithm in distinguishing the above two distributions is a negligible function of the security parameter λ . The hardness of the RLWE problem can be reduced to hard problems on ideal lattices and has been adopted as a foundational assumption in NIST's post-quantum cryptography standardization process.

2.2.3. Discrete Gaussian Distribution

Definition 4 (Discrete Gaussian Distribution). For parameter $\sigma > 0$, the discrete Gaussian distribution over the integers $\mathcal{D}_{\mathbb{Z}, \sigma}$ is defined as follows: for any $x \in \mathbb{Z}$,

$$\Pr[\mathcal{D}_{\mathbb{Z}, \sigma} = x] \propto \exp(-\pi x^2 / \sigma^2). \quad (2)$$

This distribution is used to generate the error terms in the RLWE problem; the parameter $\sigma = 3.2$ combined with $n = 1024$ and $q = 12,289$ guarantees a 128-bit post-quantum security level.

2.3. Number Theoretic Transform

Definition 5 (NTT Transform). Let q be a prime with $q \equiv 1 \pmod{2n}$, so that a primitive $2n$ -th root of unity $\omega \in \mathbb{Z}_q^*$ exists. For a polynomial $a(x) = \sum_i a_i x^i \in R_q$, the forward NTT is defined as

$$\hat{a}_j = \sum_i a_i \omega^{ij} \pmod{q}, \quad (3)$$

and the inverse NTT as

$$a_i = n^{-1} \sum_j \hat{a}_j \omega^{-ij} \pmod{q}. \quad (4)$$

The Number Theoretic Transform (NTT) is the discrete Fourier transform over a finite field, capable of reducing the computational complexity of polynomial multiplication from $O(n^2)$ to $O(n \log n)$, and is a key technique for achieving efficient lattice-based searchable encryption in this framework.

For multiplication in R_q , negative wrapped convolution is employed: by introducing $\psi = \sqrt{\omega}$ and pre-multiplying and post-dividing by powers of ψ , efficient reduction modulo $x^n + 1$ is achieved. The product of two polynomials can be computed in three steps: (1) forward NTT; (2) pointwise multiplication; (3) inverse NTT. Performance can be further

improved in practice through the use of precomputed root-of-unity tables, bit-reversal permutations, and Montgomery reduction.

2.4. Attribute-Based Encryption

Definition 6 (CP-ABE Construction). A CP-ABE construction consists of four algorithms: $\text{Setup}(1^\lambda, U)$ outputs public parameters and a master key; $\text{KeyGen}(\text{msk}, S)$ outputs a user private key; $\text{Encrypt}(\text{pp}, P, M)$ outputs a ciphertext; $\text{Decrypt}(\text{sk}_S, CT)$ outputs the plaintext when S satisfies P . BL-ABSE employs a lattice-based CP-ABE construction whose security is based on the LWE/RLWE hardness assumption.

Attribute-Based Encryption (ABE) is a public-key encryption mechanism supporting fine-grained access control. BL-ABSE adopts Ciphertext-Policy Attribute-Based Encryption (CP-ABE), which abstracts user identities as attribute sets S and encrypts data under an access policy P ; only when $P \subseteq S$ can the user decrypt the ciphertext.

2.5. Blockchain and Smart Contracts

Blockchain technology [32] provides BL-ABSE with a decentralized trust foundation and tamper-proof auditing capabilities. This framework adopts a consortium blockchain architecture, which offers the advantages of controlled access, efficient consensus, and privacy protection.

Definition 7 (Smart Contract). A smart contract is a deterministic state transition function deployed on the blockchain:

$$\text{SC} : (\text{State}, \text{Input}) \rightarrow (\text{State}', \text{Output}), \quad (5)$$

whose execution is guaranteed to be deterministic and consistent by the consensus mechanism. In BL-ABSE, smart contracts fulfill three core functions: access policy management, permission revocation control, and audit log recording.

Definition 8 (Threshold Secret Sharing). A (t, n) threshold protocol splits a secret s into n shares such that: any t or more shares can reconstruct the secret s ; any fewer than t shares reveal no information about s in the information-theoretic sense. BL-ABSE employs Shamir Secret Sharing to implement distributed management of the master key.

Theorem 1 (PBFT Fault Tolerance Bound). In a PBFT system with n nodes, the maximum number of malicious nodes that can be tolerated is

$$f = \lfloor (n - 1)/3 \rfloor. \quad (6)$$

BL-ABSE is based on a PBFT consensus variant of Hyperledger Fabric; in the typical (3,5) configuration, it can tolerate 1 malicious node.

2.6. Commitment Schemes

A commitment scheme is a core cryptographic primitive for searchable encryption. BL-ABSE employs an RLWE-based commitment construction to build the keyword index.

Construction 1 (RLWE Commitment). Let the public polynomial $a \in R_q$ be deterministically generated from a seed. For a keyword w , the commitment is computed as

$$\text{commit} = a \cdot s + H(w) \bmod q, \quad (7)$$

where s is deterministically derived via $H_{\text{seed}}(w)$. The key design insight is that s is derived from the keyword by a public hash function rather than depending on the user's private key; this guarantees that identical keywords produce identical commitments, which is the correctness guarantee for search matching. In terms of security, hiding is reduced to the RLWE decision problem, and binding is reduced to the SIS problem, as shown below.

Theorem 2 (Binding Property). *The RLWE commitment scheme satisfies computational binding under the SIS assumption, even when s is deterministically derived from w via H_{seed} .*

Proof. Suppose an adversary $\mathcal{A}$ can find two distinct keywords $w \neq w'$ such that $\text{commit}(w) = \text{commit}(w')$, i.e.,

$$a \cdot s + H(w) = a \cdot s' + H(w') \bmod q, \quad (8)$$

where $s = \text{GaussianSample}(H_{\text{seed}}(w), \sigma)$ and $s' = \text{GaussianSample}(H_{\text{seed}}(w'), \sigma)$. Rearranging yields $a \cdot (s - s') = H(w') - H(w) \bmod q$. Let $\mathbf{z} = s - s'$; since both s and s' are drawn from discrete Gaussian distributions with parameter $\sigma = 3.2$, $\mathbf{z}$ is a short polynomial with $\|\mathbf{z}\|_\infty \leq O(\sigma \sqrt{\log n})$. If $w \neq w'$, then by the collision resistance of H , $H(w) \neq H(w')$ with overwhelming probability, so the right-hand side is non-zero. This means $\mathcal{A}$ has found a short non-zero vector $\mathbf{z}$ satisfying $a \cdot \mathbf{z} = H(w') - H(w) \bmod q$, which constitutes a solution to an instance of the Ring-SIS problem. Under the SIS assumption, no PPT (or QPT) adversary can find such a solution with non-negligible probability.

Note that although s is deterministically derived, the adversary cannot freely choose s ; selecting a keyword w determines s via the composition $H_{\text{seed}} \rightarrow \text{GaussianSample}$. The collision resistance of H_{seed} ensures that distinct keywords produce distinct seeds, and the discrete Gaussian sampling process ensures that the resulting secret polynomials are short. Therefore, the adversary's only strategy is to find an algebraic relation among a , H , and H_{seed} that yields a collision, which reduces to breaking either the SIS assumption or the collision resistance of H_{seed} . $\square$

Remark on Deterministic Derivation. The use of a public hash function H_{seed} for deriving s introduces a design trade-off: it enables non-interactive search matching (any party knowing keyword w can reproduce the commitment) at the cost of making commitments deterministic, which leaks the search pattern (identical keywords produce identical commitments). This is a fundamental correctness–privacy trade-off; the privacy implications are analyzed quantitatively in Section 5.9.

2.7. Security Model

This paper employs the leakage function framework for security modeling. The leakage function of a searchable encryption protocol $\mathcal{L} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Encrypt}}, \mathcal{L}_{\text{Search}})$ characterizes the information leakage at each phase, including the search pattern $\text{sp}(w)$ and the access pattern $\text{ap}(w)$.

Definition 9 ($\mathcal{L}$ -IND-INDEX Security). *A protocol satisfies index indistinguishability if for any PPT (or QPT) adversary, its advantage in the following game is negligible: the adversary submits a challenge keyword pair (w_0^*, w_1^*) , the challenger randomly selects b and returns the encrypted index of w_b^* , and the adversary's advantage in guessing b is*

$$\text{Adv} = |\Pr[b' = b] - 1/2|. \quad (9)$$

Definition 10 ($\mathcal{L}$ -IND-TRAPDOOR Security). *A protocol satisfies query privacy if the adversary's advantage in distinguishing search tokens for two different keywords is negligible.*

Definition 11 (Post-Quantum Security). *The above security definitions remain valid when “PPT adversary” is replaced with “quantum polynomial-time adversary.” Since BL-ABSE is based on the RLWE and SIS problems, for which no polynomial-time quantum solving algorithm is currently known, the framework possesses post-quantum security.*

3. Problem Formalization

This section provides a formal model of the BL-ABSE framework. It begins with a description of the system model, clearly identifying the participating entities and their interactions. It then analyzes the threat model, characterizing adversary capabilities and security threats. Next, it presents the design objectives. Finally, it gives the formal definition of the protocol, laying the foundation for subsequent design and analysis.

3.1. System Model

The BL-ABSE framework adopts a blockchain-enhanced edge-cloud collaborative architecture, aimed at providing efficient encrypted search services for sensitive data in distributed environments, while ensuring decentralized trust and operational auditability. The system architecture diagram is shown in Figure 1.

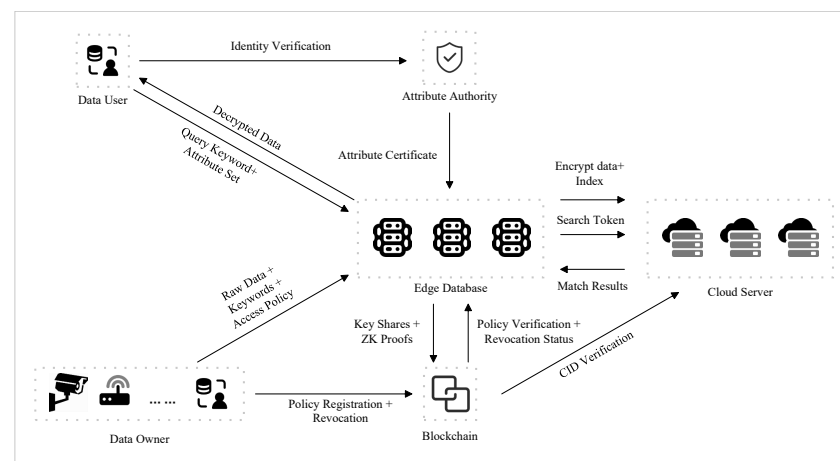


Figure 1. System architecture diagram.

3.1.1. Participating Entities

The system consists of six types of entities.

Data Owner (DO): The producer and owner of data. The DO is responsible for submitting raw data and keywords to the edge node consortium for encryption processing and for specifying an access policy for each data item. The DO must register access policies on the blockchain to support subsequent dynamic updates and permission revocation.

Data User (DU): The retriever and consumer of data. The DU first applies for an attribute certificate from the attribute authority, then obtains a private key associated with its attributes through threshold collaboration among the edge node consortium. When searching, the DU generates a search token and submits a query request; the system verifies the DU's access rights via the blockchain and returns matching results.

Edge Node Association (ENA): The core improvement of BL-ABSE. The single edge server in the original approach is replaced by an association of n edge nodes employing a (t, n) threshold mechanism. The master key is split into n shares, each held by a separate node. Any t honest nodes can collaboratively complete sensitive operations such as user key generation, while any fewer than t nodes cannot obtain any information about the master key, thereby eliminating single-point-of-trust risks.

Cloud Storage Node (CSN): Provides ciphertext storage and retrieval services. BL-ABSE uses IPFS distributed storage, where ciphertexts are indexed by content-addressed hashes (CIDs). CIDs are registered on the blockchain to ensure data integrity and traceability. The CSN is assumed to be honest-but-curious—it follows protocol execution but may attempt to infer sensitive information from the data.

Blockchain Network (BCN): A newly added core infrastructure in BL-ABSE. A consortium blockchain based on Hyperledger Fabric performs three core functions: on-chain registration, verification, and updating of access policies; maintenance and immediate enforcement of user revocation lists; and tamper-proof audit logs of all critical operations.

Attribute Authority (AA): Responsible for verifying user identities and issuing attribute certificates. The AA maintains user-to-attribute mappings to ensure that only legitimate users can obtain correct attribute claims.

3.1.2. System Architecture

BL-ABSE adopts a dual-layer architecture of “encrypted search layer + blockchain enhancement layer”. The core idea is: while maintaining the efficiency of the original encrypted search mechanism, the blockchain layer systematically strengthens the trust model and dynamic management capabilities, with clear boundaries of responsibility between the two layers.

The **encrypted search layer** handles core cryptographic operations and contains two sub-layers: (1) Keyword search sub-layer: Based on the RLWE problem, a keyword commitment mechanism is constructed. For keyword w , the system generates commitment

$$\text{commit} = a \cdot s + H(w) \bmod q, \quad (10)$$

where a is a public polynomial, s is a secret polynomial deterministically derived by the public hash function $H_{\text{seed}}(w)$, and NTT optimization achieves $O(n \log n)$ complexity. (2) Access control sub-layer: Lattice-based CP-ABE is adopted for fine-grained access control, and the data key K_{data} is encrypted by ABE under policy P .

The **blockchain enhancement layer** provides decentralized trust and dynamic management capabilities, comprising three core mechanisms: (1) Decentralized key management: The master key msk is split via (t, n) threshold secret sharing and distributed among n edge nodes; user key generation requires collaboration from at least t nodes. (2) Smart contract access control: Access policies and revocation lists are stored on the blockchain, enabling dynamic policy updates and immediate revocation via smart contracts. (3) Tamper-proof audit log: All critical operations are recorded on the blockchain, supporting retrospective tracing and accountability.

The storage architecture employs an on-chain/off-chain hybrid strategy: encrypted ciphertexts are stored on the IPFS network (off-chain), with only the content identifier CID (approximately 46 bytes) and metadata index registered on-chain. This ensures data integrity and traceability while reducing on-chain storage costs.

3.1.3. Workflow

The complete workflow of BL-ABSE includes five phases: system initialization, data upload, key request, search query, and result decryption.

Phase 1—System Initialization. The edge node consortium collaboratively generates master key shares via a Distributed Key Generation (DKG) protocol, and each node submits share verification information to the blockchain. Three smart contracts—KeyManagement, AccessControl, and AuditLog—are deployed, along with initialization of RLWE public parameters and hash functions H and H_{seed} . This phase is executed only once at system deployment.

Phase 2—Data Upload. The DO specifies keyword w and access policy P for data M , and submits them to the edge nodes. The edge nodes: generate a data key and encrypt the data with AES-GCM; derive secret polynomial s via $H_{\text{seed}}(w)$ and compute the RLWE commitment for keyword w ; encrypt the data key with CP-ABE under policy P ; store the encrypted data on IPFS to obtain the CID; and invoke the AccessControl contract to register the policy hash and CID.

Phase 3—Key Request. The DU applies to the AA for attribute certificate cert_S , then requests a private key from the edge node consortium. After at least t nodes verify cert_S , each computes a partial key and generates a zero-knowledge proof, which is submitted to the KeyManagement contract. The contract verifies all proofs and aggregates them via Lagrange interpolation to produce the complete user private key $\text{sk}_S = (S, \text{sk}_{\text{abe}, S})$.

Phase 4—Search Query. The DU derives secret polynomial s via $H_{\text{seed}}(w)$ and generates search token $T_w = (\text{commit}_q, S, ts, \sigma_{\text{user}})$ for keyword w . Since H_{seed} is a public parameter, the DU uses the same derivation method as the DO, ensuring the same keyword generates the same commitment. The system performs triple verification: verifying the token signature and validity period; invoking the AccessControl contract to check policy validity and revocation status; and executing RLWE commitment matching.

Phase 5—Result Decryption. The DU retrieves matching ciphertexts from IPFS, uses private key sk_S to decrypt the ABE ciphertext and obtain the data key, then uses AES-GCM decryption to recover the plaintext data. If the user's attributes do not satisfy the policy, ABE decryption returns $\perp$, ensuring enforcement of access control.

3.2. Threat Model

This section analyzes the security threats facing BL-ABSE, including adversary capabilities, specific threats, and trust assumptions.

3.2.1. Adversary Capabilities

This paper considers the following four types of adversaries, covering the full threat spectrum from external attacks to insider threats:

External Adversary ($\mathcal{A}_{\text{ext}}$): An attacker who does not participate in system operations. This adversary can eavesdrop on public communication channels and observe encrypted indices and ciphertexts, with the goal of inferring keywords or data content from public information. This paper considers external adversaries with quantum computing capabilities.

Malicious Cloud Storage Node ($\mathcal{A}_{\text{csn}}$): Modeled as honest-but-curious. It follows the protocol for storage and retrieval operations but may record metadata such as access patterns and search frequencies, attempting to infer sensitive information.

Malicious Data User ($\mathcal{A}_{\text{du}}$): A legitimate user attempting to access data beyond their authorized permissions. Attack methods include forging or tampering with attribute certificates, replaying expired search tokens, and colluding with other users to escalate privileges.

Byzantine Edge Node ($\mathcal{A}_{\text{byz}}$): Some edge nodes that are compromised or actively malicious. They may refuse to participate in key generation protocols, submit incorrect partial keys, record protocol interaction information to infer keys, or collude with other malicious nodes to reconstruct the master key.

3.2.2. Threat Analysis

Based on the adversary capabilities described above, the main security threats facing the system are shown in Table 2.

Table 2. Security threat analysis.

Threat ID	Threat Description	Threat Source	Potential Impact
T1	Keyword privacy disclosure	$\mathcal{A}_{\text{ext}}, \mathcal{A}_{\text{csn}}$	Adversary infers search content
T2	Data content disclosure	$\mathcal{A}_{\text{ext}}, \mathcal{A}_{\text{csn}}$	Adversary obtains plaintext data
T3	Access control bypass	$\mathcal{A}_{\text{du}}$	Unauthorized access to sensitive data
T4	Master key leakage	$\mathcal{A}_{\text{byz}}$	Complete system compromise
T5	Audit log tampering	$\mathcal{A}_{\text{byz}}, \mathcal{A}_{\text{du}}$	Inability to trace accountability
T6	Denial of service	$\mathcal{A}_{\text{byz}}$	Reduced system availability
T7	Quantum attack	$\mathcal{A}_{\text{ext}}$ (QPT)	Failure of classical cryptographic constructions

3.2.3. Trust Assumptions

Assumption 2 (Honest Majority). *At least $2n/3$ of the nodes in the edge node consortium are honest, i.e., the number of malicious nodes $f < n/3$. This is a necessary condition for the correctness of PBFT consensus. In a consortium blockchain setting, nodes are operated by pre-vetted organizations bound by commercial or legal constraints, making large-scale collusion unlikely.*

Assumption 3 (Threshold Security). *Under the (t, n) threshold configuration, the adversary can compromise at most $t - 1$ nodes. The threshold parameter t can be configured flexibly according to security requirements; a typical configuration of $(3, 5)$ allows at most 2 nodes to be compromised.*

Assumption 4 (Secure Communication). *Inter-node communication is encrypted via TLS; adversaries cannot tamper with or forge inter-node messages.*

Assumption 5 (Trusted Attribute Authority). *The attribute authority correctly performs identity verification and attribute issuance, and will not issue false attribute certificates to unauthorized users.*

Remark 1 (AA Trust Limitation). *The trust placed in the AA represents a deliberate design choice: BL-ABSE decentralizes key management (via threshold secret sharing across the edge-node consortium) but retains a centralized attribute authority for identity-to-attribute binding. This is because attribute certification inherently requires an authoritative identity-verification process (e.g., verifying an employee's department affiliation), which cannot be fully decentralized without introducing a separate decentralized identity layer. If the AA is compromised, it could issue fraudulent attribute certificates, potentially enabling unauthorized access. Several mitigation strategies exist: (i) deploying multiple independent AAs with cross-verification (multi-authority ABE); (ii) recording attribute issuance events on the blockchain audit log to enable post-hoc detection of anomalous issuance; (iii) requiring periodic AA re-certification with smart-contract-enforced expiration. The extension to a fully decentralized attribute management framework is an important direction for future work.*

Assumption 6 (Cryptographic Assumptions). *The RLWE problem and the SIS problem remain hard under the quantum computation model; the hash functions and symmetric encryption used satisfy standard security assumptions.*

3.3. Design Objectives

Based on the system model and threat analysis above, the BL-ABSE framework must satisfy the following design objectives:

Objective G1—Post-Quantum Security: The security of the framework is based on lattice hard problems (RLWE/SIS), remaining secure even against adversaries with

quantum computing capabilities. The parameter selection ($n = 1024$, $q = 12,289$) ensures a 128-bit post-quantum security level.

Objectives G2–G4: Data confidentiality ensures that unauthorized entities cannot obtain plaintext data; search privacy protection ensures that search tokens do not reveal keyword content; fine-grained access control supports attribute-based access policy definition and dynamic updates.

Objective G5—Computational Efficiency: The computational complexity for keyword commitment generation and search matching is $O(n \log n)$, and blockchain overhead is concentrated on low-frequency operations.

Objective G6—Decentralized Trust: A multi-party collaborative trust model is achieved through (t, n) threshold key management and blockchain consensus, tolerating at most $\lfloor (n - 1)/3 \rfloor$ malicious nodes.

Objectives G7–G8: All critical operations are recorded on a tamper-proof blockchain, supporting auditing and compliance review; permission revocation is executed via blockchain transactions and takes effect within approximately 2 s, without requiring data re-encryption or key redistribution.

3.4. Formal Definition of the Protocol

This section presents the formal definition of the BL-ABSE protocol, including the algorithm interfaces, correctness, and security definitions.

3.4.1. Algorithm Definitions

Definition 12 (BL-ABSE Protocol). A blockchain-enhanced attribute-based searchable encryption protocol, BL-ABSE, consists of the following nine probabilistic polynomial-time algorithms:

- $\text{BC-Setup}(1^\lambda, n, t, U) \rightarrow (\text{pp}, \{\text{sk}_i\}_{i \in [n]}, \text{BC})$: System initialization algorithm. Takes as input security parameter λ , number of edge nodes n , threshold parameter t , and attribute universe U . Outputs public parameters pp (including RLWE parameters, hash functions H and H_{seed} , ABE public parameters, etc.), key shares $\{\text{sk}_i\}$ for each node, and the initialized blockchain state BC .
- $\text{ThresholdKeyGen}(\text{pp}, \{\text{sk}_i\}_{i \in T}, S) \rightarrow \text{sk}_S$: Threshold user key generation algorithm. Takes as input public parameters pp , key shares of the participating node set T ($|T| \geq t$), and user attribute set S . At least t nodes collaborate; each computes a partial key and aggregates via the blockchain to output user private key $\text{sk}_S = (S, \text{sk}_{\text{abe}, S})$.
- $\text{BC-Encrypt}(\text{pp}, w, P, M) \rightarrow (\text{CID}, I, \text{txHash})$: Data encryption algorithm. Takes as input public parameters pp , keyword w , access policy P , and plaintext M . Derives secret polynomial s via $H_{\text{seed}}(w)$ and computes a keyword commitment. Outputs IPFS content identifier CID , searchable index I , and the blockchain transaction hash txHash of the policy registration.
- $\text{Trapdoor}(\text{pp}, \text{sk}_S, w) \rightarrow T_w$: Search token generation algorithm. Takes as input public parameters pp , user private key sk_S , and query keyword w . Derives the same secret polynomial s as the encryption side via $H_{\text{seed}}(w)$, and outputs search token $T_w = (\text{commit}_q, S, ts, \text{expiry}, \sigma_{\text{user}})$. Since H_{seed} is a public parameter, the same keyword produces the same commitment during both encryption and searching.
- $\text{BC-Search}(T_w, \text{BC}, \text{DB}) \rightarrow R$: Search algorithm. Takes as input search token T_w , blockchain state BC , and encrypted database DB . Performs triple verification—token validation, on-chain permission check, and commitment matching—and outputs the set of matching results R .
- $\text{Decrypt}(\text{pp}, \text{sk}_S, R) \rightarrow M$: Decryption algorithm. Takes as input public parameters pp , user private key sk_S , and search result set R . For each result, performs ABE decryption

to obtain the data key, then AES decryption to recover the plaintext, and outputs plaintext set M .

- $\text{RevokeAccess}(\text{pp}, \text{dataId}, \text{userId}, \text{reason}) \rightarrow \text{txHash}$: Permission revocation algorithm. The data owner invokes this algorithm to add the specified user to the revocation list. The revocation is executed via a blockchain transaction and takes effect immediately upon confirmation.
- $\text{UpdatePolicy}(\text{pp}, \text{dataId}, P_{\text{new}}, \text{reason}) \rightarrow \text{txHash}$: Policy update algorithm. The data owner invokes this algorithm to update the access policy. The new policy hash replaces the old value on-chain and immediately affects all subsequent search requests.
- $\text{QueryAuditLog}(\text{dataId}, \text{timeRange}) \rightarrow \text{logs}$: Audit log query algorithm. Queries all operation records for the specified data item within the given time range, returning a tamper-proof list of audit logs.

3.4.2. Correctness Definition

Definition 13 (Correctness). The BL-ABSE protocol is correct if, for all security parameters λ , all keywords $w \in \{0, 1\}^*$, all access policies P , all attribute sets S satisfying $P \subseteq S$, and all plaintexts M , the following equation holds with overwhelming probability:

$$\text{Decrypt}(\text{pp}, \text{sk}_S, \text{BC-Search}(\text{Trapdoor}(\text{pp}, \text{sk}_S, w), \text{BC}, \text{DB})) = M, \quad (11)$$

where DB contains the encryption result of $\text{BC-Encrypt}(\text{pp}, w, P, M)$, $\text{sk}_S \leftarrow \text{ThresholdKeyGen}(\text{pp}, \{\text{sk}_i\}, S)$, and the user has not been revoked. Correctness encompasses three sub-conditions: (1) Commitment matching correctness: The same keyword w derives the same secret polynomial s via the public hash function H_{seed} , producing the same commitment. (2) ABE decryption correctness: When $P \subseteq S$, ABE.Decrypt correctly recovers the data key. (3) Threshold aggregation correctness: When $|T| \geq t$, Lagrange interpolation correctly recovers the complete private key.

3.4.3. Security Definitions

The security of BL-ABSE is characterized by the following formal definitions:

Definition 14 ($\mathcal{L}$ -IND-INDEX Security). BL-ABSE satisfies index indistinguishability if, for any PPT (or QPT) adversary $\mathcal{A}$, the adversary's advantage in the following security game is negligible: the challenger runs BC-Setup ; the adversary may issue polynomially many encryption queries and search token queries; the adversary submits a challenge keyword pair (w_0^*, w_1^*) and policy P^* ; the challenger randomly selects $b \leftarrow \{0, 1\}$ and returns the encrypted index I^* of w_b^* ; the adversary outputs a guess b' . The adversary's advantage is defined as

$$\text{Adv}^{\text{IND-INDEX}}(\mathcal{A}) = |\Pr[b' = b] - 1/2|. \quad (12)$$

Definition 15 ($\mathcal{L}$ -IND-TRAPDOOR Security). BL-ABSE satisfies query privacy if, for any PPT (or QPT) adversary $\mathcal{A}$, its advantage in distinguishing search tokens T_{w_0} and T_{w_1} for two different keywords w_0, w_1 is negligible.

Definition 16 (Threshold Key Security). Under the (t, n) threshold configuration, for any PPT adversary $\mathcal{A}$ that compromises at most $t - 1$ edge nodes, its advantage in recovering the complete master key msk is negligible.

Definition 17 (Byzantine Fault Tolerance). BL-ABSE satisfies Byzantine fault tolerance if, with at most $f = \lfloor (n - 1)/3 \rfloor$ malicious edge nodes, the system still guarantees: (1) Safety: honest nodes will not accept invalid key shares or transactions; (2) Liveness: legitimate requests will eventually be processed.

Definition 18 (Audit Log Tamper-Proofness). For any PPT adversary $\mathcal{A}$, the probability of modifying historical audit logs without detection is negligible.

Definition 19 (Revocation Immediacy). If user u is revoked access to data d at time t , then for any search request at time $t' > t + \Delta_{\text{confirm}}$ (where $\Delta_{\text{confirm}} \approx 2$ seconds is the blockchain confirmation delay), user u cannot obtain data d .

4. Protocol Design

This section provides a detailed description of the concrete construction of the BL-ABSE framework. It begins with an overview of the overall design philosophy and architecture, then sequentially describes the design details of each core algorithm in the order of system initialization, threshold key generation, data encryption, search token generation, search matching, and result decryption. It then describes the dynamic permission management and audit query mechanisms, presents the smart contract design, and finally conducts correctness analysis and complexity analysis. The design of each algorithm strictly corresponds to the formal definitions in Section 3.4. The specific flow of the protocol is shown in Figure 2.

4.1. Protocol Overview

The design of BL-ABSE follows two core principles: (1) Separation of cryptographic and management layers—the cryptographic construction of the encrypted search layer is independent of the blockchain layer, ensuring that post-quantum security is unaffected by blockchain components; (2) Lightweight high-frequency operations—blockchain interactions are concentrated in low-frequency operations (system initialization, key generation, policy registration), so that high-frequency search operations only involve local computation and cached queries.

The encrypted search layer contains two sub-layers. The keyword search sub-layer exploits the hardness of the RLWE problem to construct keyword commitments: for keyword w , the system generates commitment

$$\text{commit} = a \cdot s + H(w) \bmod q, \quad (13)$$

where a is a public polynomial and s is a secret polynomial deterministically derived from keyword w via hash function H_{seed} . Using NTT optimization, the computational complexity of commitment generation is $O(n \log n)$. The access control sub-layer adopts lattice-based CP-ABE to achieve fine-grained access control, with data key K_{data} encrypted by ABE under policy P .

The blockchain enhancement layer contains three core mechanisms: (1) Decentralized key management—master key msk is distributed among n edge nodes via (t, n) threshold secret sharing; (2) Smart contract access control—access policies and revocation lists are managed on-chain, supporting dynamic updates and immediate revocation; (3) Tamper-proof audit log—all critical operations are recorded on the blockchain, supporting retrospective tracing.

The following sections describe the concrete construction of each algorithm in the order of the workflow.

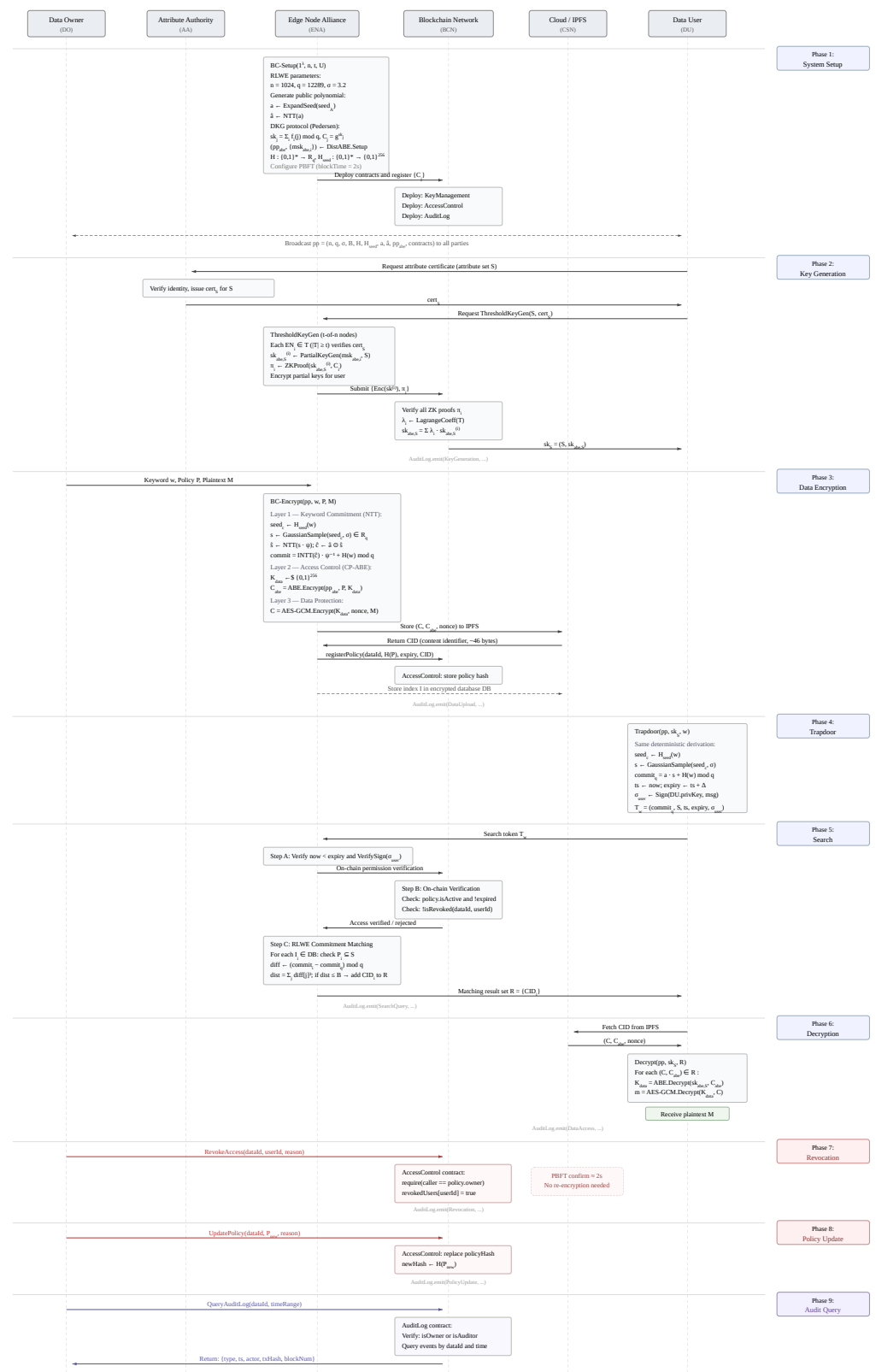


Figure 2. System flow diagram. The asterisk (*) indicates steps that are conditionally executed based on cache state.

4.2. System Initialization

The system initialization algorithm BC-Setup is executed collaboratively by the edge node consortium, completing four tasks: RLWE parameter generation, blockchain deploy-

ment, distributed key generation, and ABE initialization. Algorithm 1 is executed only once at system deployment, corresponding to the BC-Setup interface in Definition 12.

Algorithm 1 BC-Setup($1^\lambda, n, t, U$)

Require: Security parameter λ , number of edge nodes n , threshold t , attribute universe U

Ensure: Public parameters pp , node shares $\{sk_i\}$, blockchain state BC

```

1: RLWE parameter selection:  $n_{\text{lattice}} \leftarrow 1024, q \leftarrow 12,289, \sigma \leftarrow 3.2, B \leftarrow 40$ 
2:  $\text{seed}_A \xleftarrow{\$} \{0,1\}^{256}$ 
3:  $a \leftarrow \text{ExpandSeed}(\text{seed}_A) \in R_q$ 
4:  $\hat{a} \leftarrow \text{NTT}(a)$ 
5:  $BC \leftarrow \text{InitFabricNetwork}(n)$ 
6:  $\text{ConfigurePBFT}(\text{blockTime} = 2 \text{ s})$ 
7:  $\text{addr}_{\text{KM}} \leftarrow \text{DeployContract}(\text{KeyManagement})$ 
8:  $\text{addr}_{\text{AC}} \leftarrow \text{DeployContract}(\text{AccessControl})$ 
9:  $\text{addr}_{\text{AL}} \leftarrow \text{DeployContract}(\text{AuditLog})$ 
10: for  $i = 1$  to  $n$  do
11:    $f_i(x) \xleftarrow{\$} \mathbb{Z}_q[x], \deg(f_i) = t - 1$ 
12:   for  $j = 1$  to  $n$  do
13:      $s_{i,j} \leftarrow f_i(j)$ 
14:      $\text{SecureSend}(\text{EN}_i \rightarrow \text{EN}_j, s_{i,j})$ 
15:   end for
16:   for  $j = 1$  to  $n$  do
17:      $sk_j \leftarrow \sum_i s_{i,j} \bmod q$ 
18:      $C_j \leftarrow g^{sk_j}$ 
19:      $\text{KeyManagement.registerNode}(\text{EN}_j.\text{addr}, C_j)$ 
20:   end for
21: end for
22:  $(pp_{\text{abe}}, \{msk_{\text{abe},i}\}) \leftarrow \text{DistributedABE.Setup}(1^\lambda, U, t, n)$ 
23:  $pp \leftarrow (n, q, \sigma, B, H, H_{\text{seed}}, a, \hat{a}, pp_{\text{abe}}, \text{contracts})$ 
24: return  $(pp, \{sk_i\}, BC)$ 

```

Design Notes. (1) Parameter selection: Lattice dimension $n = 1024$ ensures a 128-bit post-quantum security level; prime $q = 12,289$ satisfies the NTT requirement ($q \equiv 1 \bmod 2n$) and is the smallest prime satisfying this condition, helping to reduce computational overhead; standard deviation $\sigma = 3.2$ achieves a balance between security and correctness; threshold $B = 40$ is determined by noise analysis, as detailed in Section 4.11. (2) Precomputation optimization: Step 4 precomputes the NTT representation of a and stores it; all subsequent commitment generation uses this representation directly, avoiding repeated forward NTT transforms. (3) DKG protocol: Pedersen DKG is employed; each node generates a random degree- $(t - 1)$ polynomial and distributes shares to other nodes, so that each node's final share sk_j is the sum of evaluations of all polynomials at point j . The public verification commitment $C_j = g^{sk_j}$ is registered on-chain to support subsequent zero-knowledge proof verification. (4) Hash function configuration: The public parameters include two independent hash functions $H : \{0,1\}^* \rightarrow R_q$ (for keyword encoding) and $H_{\text{seed}} : \{0,1\}^* \rightarrow \{0,1\}^{256}$ (for deriving Gaussian sampling seeds); their independence is key to the security proof.

Parameter Security Strength Analysis. For the parameters ($n = 1024, q = 12,289, \sigma = 3.2$), we estimate the concrete security level using standard lattice-estimator methodology. The root Hermite factor is $\delta = (q/\sigma)^{1/n} \approx 1.0082$, which corresponds to a BKZ block size $\beta \approx 456$ under the classical Core-SVP model (cost $\approx 2^{0.292\beta} \approx 2^{133}$) and $\beta \approx 456$ under the quantum sieving model (cost $\approx 2^{0.265\beta} \approx 2^{121}$). Both estimates exceed the 128-bit classical (respectively, 120-bit quantum) security threshold required for NIST Level 1.

Comparison with NIST FIPS 203 (ML-KEM). The NIST ML-KEM-768 standard [13] uses parameters $n = 256, k = 3, q = 3329$ in a Module-LWE setting, yielding an effective ring dimension of 768 and achieving NIST Level 3 (≥ 192 -bit security). BL-ABSE uses a different ring structure ($R_q = \mathbb{Z}_{12,289}[x]/(x^{1024} + 1)$, ideal-lattice RLWE) with a larger ring dimension ($n = 1024$) and larger modulus ($q = 12,289$ vs. 3329). The larger modulus is required by the NTT constraint $q \equiv 1 \pmod{2n}$ and the commitment arithmetic. Despite the structural differences, both parameter sets derive their hardness from the same family of lattice problems (RLWE $\rightarrow$ approximate SVP on ideal lattices), validating the post-quantum security foundation of BL-ABSE.

4.3. Threshold User Key Generation

User key generation requires collaboration from at least t edge nodes; a single node cannot independently generate a valid key. This design eliminates the reliance on a single key generation center in traditional approaches. Algorithm 2 corresponds to the ThresholdKeyGen interface in Definition 12.

Algorithm 2 ThresholdKeyGen(pp, $S, \{EN_i\}_{i \in T}$)

Require: Public parameters pp, user attribute set S , participating node set T with $|T| \geq t$

Ensure: User private key sk_S

```

1:  $userId \leftarrow H(\text{DU.pubKey}), \text{attrHash} \leftarrow H(S)$ 
2:  $\text{requestId} \leftarrow \text{KeyManagement.requestKeyGen}(userId, \text{attrHash}, \text{nonce})$ 
3: for each  $EN_i \in T$  do
4:   if VerifyAttributeCert( $S, \text{AA.cert}$ ) then
5:      $sk_{abe,S}^{(i)} \leftarrow \text{ABE.PartialKeyGen}(\text{msk}_{abe,i}, S)$ 
6:      $\pi_i \leftarrow \text{GenerateZKProof}(sk_{abe,S}^{(i)}, C_i)$ 
7:      $\text{KeyManagement.submitPartialKey}(\text{requestId}, i, \text{Enc}(sk^{(i)}), \pi_i)$ 
8:   end if
9: end for
10: if KeyManagement.verifyAllProofs(requestId) then
11:    $\lambda_i \leftarrow \text{LagrangeCoefficients}(T)$ 
12:    $sk_{abe,S} \leftarrow \sum_{i \in T} \lambda_i \cdot sk_{abe,S}^{(i)}$ 
13: end if
14:  $sk_S \leftarrow (S, sk_{abe,S})$ 
15: AuditLog.emit(KeyGeneration, userId, attrHash,  $T, ts$ )
16: return  $sk_S$ 
```

Design Notes. (1) Zero-knowledge proofs: In Step 6, each node generates a zero-knowledge proof π_i , proving that its submitted partial key was correctly computed from its held share sk_i , without leaking sk_i itself. This prevents Byzantine nodes from submitting incorrect shares to corrupt key correctness. (2) Encrypted transmission: In Step 7, partial keys are submitted to the chain in encrypted form, decryptable only by the requesting user, preventing other nodes or on-chain observers from obtaining partial keys. (3) Simplified private key structure: Unlike traditional approaches, the user private key sk_S in this framework consists only of attribute set S and ABE private key $sk_{abe,S}$, without any user-specific PRF keys. This is because the deterministic derivation of keyword commitments relies entirely on the public-parameter hash function H_{seed} , not on user-private keys, thereby ensuring that different users generate the same search commitment for the same keyword, enabling correct search matching.

4.4. Data Encryption

The data encryption algorithm BC-Encrypt is one of the core algorithms of BL-ABSE. It executes five steps: keyword commitment generation (NTT-optimized), ABE encryption,

symmetric encryption, IPFS storage, and on-chain registration. Algorithm 3 corresponds to the BC-Encrypt interface in Definition 12.

Algorithm 3 BC-Encrypt(pp, w , P , M)

Require: Public parameters pp, keyword w , access policy P , plaintext M

Ensure: CID, index I , transaction hash txHash

```

1:  $\text{seed}_c \leftarrow H_{\text{seed}}(w)$ 
2:  $s \leftarrow \text{GaussianSample}(\text{seed}_c, \sigma) \in R_q$ 
3:  $e_w \leftarrow H(w) \in R_q$ 
4:  $\hat{s} \leftarrow \text{NTT}(s \cdot \psi)$ 
5:  $\hat{c} \leftarrow \hat{a} \odot \hat{s}$ 
6:  $c \leftarrow \text{INTT}(\hat{c}) \cdot \psi^{-1}$ 
7:  $\text{commit} \leftarrow c + e_w \bmod q$ 
8:  $K_{\text{data}} \xleftarrow{\$} \{0, 1\}^{256}$ 
9:  $C_{\text{abe}} \leftarrow \text{ABE.Encrypt}(\text{pp}_{\text{abe}}, P, K_{\text{data}})$ 
10:  $\text{nonce} \xleftarrow{\$} \{0, 1\}^{96}$ 
11:  $C \leftarrow \text{AES-GCM.Encrypt}(K_{\text{data}}, \text{nonce}, M)$ 
12:  $\text{data\_blob} \leftarrow \text{Serialize}(C, C_{\text{abe}}, \text{nonce})$ 
13:  $\text{CID} \leftarrow \text{IPFS.Add}(\text{data\_blob})$ 
14:  $\text{dataId} \leftarrow H(\text{CID} \parallel \text{owner} \parallel \text{ts})$ 
15:  $\text{policyHash} \leftarrow H(P)$ 
16:  $\text{txHash} \leftarrow \text{AccessControl.registerPolicy}(\text{dataId}, \text{policyHash}, \text{expireTime}, \text{CID})$ 
17:  $\text{AuditLog.emit}(\text{DataUpload}, \text{dataId}, \text{owner}, \text{policyHash}, \text{CID}, \text{ts})$ 
18:  $I \leftarrow (\text{commit}, \text{policyHash}, \text{CID}, \text{ts})$ 
19: return (CID,  $I$ , txHash)
  
```

NTT Optimization Details. Steps 1–7 are the efficiency core of this framework. The naive implementation of polynomial multiplication $a \cdot s$ over $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ requires $O(n^2)$ modular multiplications. BL-ABSE uses NTT optimization based on the negative wrapped convolution to reduce this to $O(n \log n)$: (1) Pre-multiply by ψ (Step 4): Introduce the $2n$ -th primitive root of unity ψ (satisfying $\psi^2 = \omega$) and multiply the coefficients s_i of polynomial s by ψ^i . This preprocessing transforms the mod- $(x^n + 1)$ convolution into a standard cyclic convolution, making the NTT directly applicable. (2) Pointwise multiplication in the NTT domain (Step 5): In the NTT domain, polynomial multiplication degenerates to n independent modular multiplications, each requiring $O(1)$ time. Since $\hat{a}$ was precomputed during system initialization (Algorithm 1, Step 4), only the NTT of s and the pointwise multiplication need to be computed here. (3) Inverse transform and post-divide (Step 6): Apply INTT to the pointwise product to obtain the convolution result, then divide by powers of ψ to recover the original coordinates.

Deterministic Commitment Mechanism. The key design in this construction is Step 1: the seed seed_c is derived directly from keyword w by the public hash function H_{seed} , rather than relying on user-private keys. This design ensures: (i) the same keyword w , regardless of who encrypts or searches, always produces the same seed $\text{seed}_c = H_{\text{seed}}(w)$; (ii) which in turn derives the same secret polynomial s ; (iii) ultimately producing the same commitment. This is the fundamental guarantee of search matching correctness.

On-chain/Off-chain Separation Design. Encrypted data are stored on IPFS (off-chain); only the CID (46 bytes), policy hash (32 bytes), and a small amount of metadata are stored on-chain (Steps 14–16). The rationale is: (1) On-chain storage costs far exceed off-chain costs, making it uneconomical to put large amounts of ciphertext on-chain; (2) IPFS's content-addressing mechanism naturally ensures data integrity—the CID is a cryptographic hash of the data, and tampering with the data will change the CID; (3) Once the CID is registered on-chain, it is immutable, ensuring data traceability.

4.5. Search Token Generation

The search token generation algorithm Trapdoor is executed locally by the data user. It generates a search token containing a commitment, attributes, timestamp, and signature. Token generation does not involve blockchain interaction, ensuring low-latency search requests. Algorithm 4 corresponds to the Trapdoor interface in Definition 12.

Algorithm 4 Trapdoor(pp, sk_S, w)

Require: Public parameters pp, user private key sk_S = (S, sk_{abe,S}), query keyword w

Ensure: Search token T_w

```

1: seedc ← Hseed(w)
2: s ← GaussianSample(seedc, σ)
3: commitq ← NTT_Multiply(a, s) + H(w) mod q
4: ts ← CurrentTimestamp()
5: expiry ← ts + TOKEN_VALIDITY
6: msg ← (commitq || S || ts || expiry)
7: σuser ← Sign(DU.privKey, msg)
8: Tw ← (commitq, S, ts, expiry, σuser)
9: return Tw

```

Design Notes. (1) Matching principle: The computation in Steps 1–3 is identical to Steps 1–7 of Algorithm 3—the same keyword w derives the same seed seed_c via the public hash function H_{seed} , then generates the same secret polynomial s and the same commitment commit_q . This is the basis for search matching correctness. Since seed derivation depends only on public parameter H_{seed} and keyword w , and not on user identity, any authorized user who knows the keyword can generate the correct search token. (2) Anti-replay mechanism: The token contains timestamp ts and validity period expiry (Steps 4–5); expired tokens will be rejected by the search algorithm. User signature σ_{user} (Step 7) binds the token content to user identity, preventing tampering and forgery. (3) Efficiency characteristics: Token generation is executed entirely locally by the user, requiring no network communication or blockchain interaction; latency is determined only by a single NTT polynomial multiplication and a single signature computation.

4.6. Search Matching

The search algorithm BC-Search is the core of BL-ABSE's secure search. It performs triple verification: token validation, on-chain permission check, and commitment matching. Only data items that pass all verification steps are added to the result set. Algorithm 5 corresponds to the BC-Search interface in Definition 12.

Triple Verification Mechanism Details. The search process of BL-ABSE fuses the cryptographic matching of traditional searchable encryption with dynamic permission checking on the blockchain, forming triple verification: (1) On-chain policy verification (Steps 5–6): Checks whether the data's access policy is still valid. A policy may be invalidated due to expiry or being explicitly disabled by the data owner. This check ensures that invalidated policies are no longer used. (2) On-chain revocation check (Step 7): Queries the revocation list on the blockchain to check whether the current user has been revoked access to the data. This is the key step for achieving immediate revocation. (3) RLWE commitment matching (Steps 9–13): This is the core cryptographic matching step. The difference between the stored commitment commit_i and the query commitment commit_q is computed, and the difference coefficients are mapped to the interval $[-q/2, q/2]$ via centered reduction (Steps 10–11). The squared Euclidean distance is then computed. If the distance is less than threshold B , a match is declared.

Algorithm 5 BC-Search(T_w , BC, DB)**Require:** Search token T_w , blockchain state BC, encrypted database DB**Ensure:** Set of matching results R

```

1: if CurrentTimestamp() > expiry then
2:   return REJECT
3: end if
4: if  $\neg$ VerifySignature(DU.pubKey, msg,  $\sigma_{\text{user}}$ ) then
5:   return REJECT
6: end if
7:  $R \leftarrow \{\}$ , userId  $\leftarrow H(\text{DU.pubKey})$ 
8: for each  $(I_i, \text{CID}_i) \in \text{DB}$  do
9:   policy  $\leftarrow$  AccessControl.getPolicy(dataId $i$ )
10:  if policy.expireTime < now or  $\neg$ policy.isActive then
11:    continue
12:  end if
13:  if AccessControl.isRevoked(dataId $i$ , userId) then
14:    continue
15:  end if
16:  if  $P_i \notin S$  then
17:    continue
18:  end if
19:  diff  $\leftarrow (\text{commit}_i - \text{commit}_q) \bmod q$ 
20:  for  $j = 0$  to  $n - 1$  do
21:    if diff[j] >  $q/2$  then
22:      diff[j]  $\leftarrow$  diff[j] -  $q$ 
23:    end if
24:  end for
25:  dist  $\leftarrow \sum_j \text{diff}[j]^2$ 
26:  if dist  $\leq B$  then
27:     $R \leftarrow R \cup \{\text{CID}_i\}$ 
28:  end if
29: end for
30: AuditLog.emit(SearchQuery, queryId, userId,  $H(\text{commit}_q)$ ,  $|R|$ , ts)
31: return  $R$ 

```

Correctness Analysis of Matching. When the search keyword w matches the data keyword, since both sides use the same public hash function $H_{\text{seed}}(w)$ to produce the same seed seed_c , and subsequently derive the same secret polynomial s , we have $\text{commit}_i - \text{commit}_q = (a \cdot s + H(w)) - (a \cdot s + H(w)) = 0$, so $\text{dist} = 0 \leq B$ and the match succeeds. When keywords differ, $H(w)$ and $H(w')$ are independently random polynomials, and the norm of the difference exceeds B with overwhelming probability, causing the match to fail.

On-chain Query Optimization. To avoid the latency overhead of on-chain queries with every search, BL-ABSE employs a local cache + event subscription mechanism: edge nodes cache policies and revocation lists locally and update the cache in real time by subscribing to blockchain events. When a policy change or revocation operation triggers a blockchain event, the cache is updated within the next block confirmation time (approximately 2 s). This reduces the 15 ms latency of on-chain queries to microsecond-level memory access.

4.7. Result Decryption

The decryption algorithm retrieves matching ciphertexts from IPFS and sequentially performs ABE decryption and AES decryption. Algorithm 6 corresponds to the Decrypt interface in Definition 12.

Algorithm 6 Decrypt(pp, sk_S, R)**Require:** Public parameters pp , user private key sk_S , result set R **Ensure:** Plaintext set M

```

1:  $M \leftarrow \{\}$ 
2: for each  $CID \in R$  do
3:    $(C, C_{abe}, nonce) \leftarrow \text{Deserialize}(\text{IPFS.Get}(CID))$ 
4:    $K_{data} \leftarrow \text{ABE.Decrypt}(pp_{abe}, sk_{abe,S}, C_{abe})$ 
5:   if  $K_{data} = \perp$  then
6:     continue
7:   end if
8:    $m \leftarrow \text{AES-GCM.Decrypt}(K_{data}, nonce, C)$ 
9:   if  $m \neq \perp$  then
10:     $M \leftarrow M \cup \{m\}$ 
11:   end if
12:    $\text{AuditLog.emit}(\text{DataAccess}, CID, \text{userId}, ts)$ 
13: end for
14: return  $M$ 

```

Design Notes. (1) Two-layer decryption: The ABE decryption in Step 4 recovers data key K_{data} ; the AES-GCM decryption in Step 6 recovers plaintext M . The two-layer separation means that access control (guaranteed by ABE) and data encryption (guaranteed by AES) operate independently, with their security not affecting each other. (2) Integrity verification: AES-GCM provides authenticated encryption; the decryption process in Step 6 simultaneously verifies ciphertext integrity. If the ciphertext has been tampered with, decryption returns $\perp$, effectively preventing ciphertext substitution attacks. (3) Audit logging: Every data access is recorded on-chain (Step 8), including the accessor, timestamp, and data identifier, supporting retrospective tracing.

4.8. Dynamic Permission Management

Dynamic permission management is an important improvement of BL-ABSE over traditional ABSE constructions. This section describes two algorithms—permission revocation and policy update—both of which take effect immediately through blockchain transactions.

4.8.1. Permission Revocation

The permission revocation algorithm allows the data owner to immediately revoke a specific user's access rights without re-encryption. Algorithm 7 corresponds to the RevokeAccess interface in Definition 12.

Algorithm 7 RevokeAccess($pp, dataId, userId, reason$)**Require:** Data ID $dataId$, revoked user ID $userId$, revocation reason $reason$ **Ensure:** Transaction hash $txHash$

```

1:  $caller \leftarrow msg.sender$ 
2:  $policy \leftarrow \text{AccessControl.getPolicy}(dataId)$ 
3: require( $caller == policy.owner$ )
4:  $txHash \leftarrow \text{AccessControl.revokeAccess}(dataId, userId)$ 
5:  $\text{AuditLog.emit}(\text{Revocation}, dataId, userId, caller, reason, ts)$ 
6: return  $txHash$ 

```

Immediate Effect Mechanism. Once the revocation transaction is submitted to the blockchain and confirmed by PBFT consensus (approximately 2 s), it is written into the tamper-proof ledger. Since BC-Search checks the revocation list on every search (Algorithm 5, Step 7), all subsequent search requests from the revoked user are immediately rejected. Compared to traditional approaches that require re-encrypting all related

data (which may take hours), the on-chain revocation list mechanism reduces the revocation delay from hours to seconds, with no re-encryption operations required.

4.8.2. Policy Update

The policy update algorithm allows the data owner to dynamically modify access policies. Algorithm 8 corresponds to the UpdatePolicy interface in Definition 12.

Algorithm 8 UpdatePolicy(pp, dataId, P_{new} , reason)

Require: Data ID dataId, new policy P_{new} , update reason reason

Ensure: Transaction hash txHash

```

1: caller  $\leftarrow$  msg.sender
2: policy  $\leftarrow$  AccessControl.getPolicy(dataId)
3: require(caller == policy.owner)
4: oldHash  $\leftarrow$  policy.policyHash
5: newHash  $\leftarrow$   $H(P_{new})$ 
6: txHash  $\leftarrow$  AccessControl.updatePolicy(dataId, newHash)
7: AuditLog.emit(PolicyUpdate, dataId, oldHash, newHash, reason, ts)
8: return txHash

```

Design Notes and Limitations. On-chain policy updates only replace the policy hash (Step 6), immediately affecting policy matching verification during subsequent searches.

4.9. Audit Query

The audit query algorithm allows authorized users to query the complete operation history, supporting compliance review and security incident investigation. Algorithm 9 corresponds to the QueryAuditLog interface in Definition 12.

Algorithm 9 QueryAuditLog(dataId, timeRange, requester)

Require: Data ID dataId, time range (start, end), requester requester

Ensure: Audit log list logs

```

1: isOwner  $\leftarrow$  (requester == AccessControl.getPolicy(dataId).owner)
2: isAuditor  $\leftarrow$  AuditLog.isAuthorizedAuditor(requester)
3: require(isOwner or isAuditor)
4: events  $\leftarrow$  AuditLog.queryEvents(dataId, start, end)
5: logs  $\leftarrow$  []
6: for each event  $\in$  events do
7:   log  $\leftarrow$  {type, timestamp, actor, details, txHash, blockNum}
8:   logs.append(log)
9: end for
10: return logs

```

BL-ABSE records six types of audit events: DataUpload, KeyGeneration, SearchQuery, DataAccess, Revocation, and PolicyUpdate. Each log entry contains the operation type, timestamp, actor identity, detailed information, and blockchain transaction hash, supporting complete operation tracing. Audit logs are stored on the blockchain, with their tamper-proofness guaranteed by the consensus mechanism.

4.10. Smart Contract Design

BL-ABSE deploys three core smart contracts, responsible for key management, access control, and audit logging, respectively. This section presents the key data structures and interface designs for each contract.

4.10.1. AccessControl Contract

The AccessControl contract is the core of BL-ABSE's on-chain governance, managing the lifecycle of access policies and the user revocation list.

Core Data Structures. The Policy struct contains: policyHash (policy hash, bytes32), owner (data owner address), createTime and expireTime (creation and expiration times), isActive (validity flag), ipfsCid (IPFS content identifier), and a revokedUsers mapping (user revocation status).

Core Interfaces. (1) registerPolicy (dataId, policyHash, expireTime, CID): Registers a new policy, called only during data upload. (2) verifyAccess (dataId, userId, attrHash): Verifies whether a user is authorized to access the specified data. (3) revokeAccess (dataId, userId): Adds a user to the revocation list. (4) isRevoked (dataId, userId): Queries a user's revocation status. (5) updatePolicy (dataId, newPolicyHash): Updates the policy hash.

4.10.2. KeyManagement Contract

The KeyManagement contract coordinates the on-chain process of threshold key generation, ensuring the correctness of partial keys submitted by each node.

Core Interfaces. (1) registerNode(nodeAddr, commitment): Registers an edge node and its public verification commitment. (2) requestKeyGen (userId, attrHash, nonce): The user initiates a key generation request. (3) submitPartialKey (requestId, nodeId, encPartialKey, proof): A node submits an encrypted partial key and a zero-knowledge proof. (4) verifyAllProofs (requestId): Verifies all submitted zero-knowledge proofs; if all pass, triggers key aggregation.

4.10.3. AuditLog Contract

The AuditLog contract records all critical operations in the system, providing a tamper-proof audit trail.

Core Interfaces. (1) emit (eventType, ...): Emits an audit event, automatically attaching a timestamp and transaction hash. (2) queryEvents (dataId, startTime, endTime): Queries audit events by data ID and time range. (3) isAuthorizedAuditor (addr): Verifies whether the requester has audit privileges. The contract uses Hyperledger Fabric's event mechanism to implement real-time notification; external systems can subscribe to specific types of audit events.

4.11. Correctness Analysis

Theorem 3 (Correctness). *BL-ABSE satisfies the correctness requirement of Definition 13. That is, for any keyword w , policy P , attribute set S satisfying $P \subseteq S$, and plaintext M , if the user has not been revoked, decryption correctly recovers the plaintext with overwhelming probability.*

Proof. Correctness depends on the simultaneous satisfaction of the following three sub-conditions:

(1) Commitment Matching Correctness. This construction uses the public hash function H_{seed} to derive seed $\text{seed}_c = H_{\text{seed}}(w)$ from keyword w . Since H_{seed} is a deterministic function included in the public parameters, the same keyword w , whether at the encryption side or the search side, always produces the same seed seed_c , and subsequently derives the same secret polynomial s via GaussianSample. Therefore, the commitment $\text{commit} = a \cdot s + H(w)$ at the encryption side strictly equals the commitment $\text{commit}_q = a \cdot s + H(w)$ at the search side, with $\text{diff} = 0$ and $\text{dist} = 0 \leq B$. For different keywords $w \neq w'$, by the randomness of hash function H , each coefficient of $H(w) - H(w')$ is approximately uniformly distributed over $\mathbb{Z}_q$, and the norm of the difference exceeds B with overwhelming probability.

(2) ABE Decryption Correctness. By the correctness guarantee of the lattice-based CP-ABE construction, when the user attribute set S satisfies the access policy P (i.e., $P \subseteq S$), $\text{ABE.Decrypt}(\text{pp}_{\text{abe}}, \text{sk}_{\text{abe}, S}, C_{\text{abe}})$ correctly recovers data key K_{data} .

(3) Threshold Aggregation Correctness. By the reconstruction property of Shamir secret sharing and the correctness of Lagrange interpolation, when the number of participating nodes $|T| \geq t$, the aggregated $\text{sk}_{\text{abe}, S}$ is equivalent to the private key generated directly from the complete master key msk_{abe} .

Determination of Threshold B . Threshold B must satisfy two conditions: (i) for matching keyword pairs, $\text{dist} \leq B$ holds with overwhelming probability; (ii) for non-matching keyword pairs, $\text{dist} > B$ holds with overwhelming probability. Since $\text{dist} = 0$ when matching (both sides derive identical commitments from the same keyword), condition (i) is satisfied trivially for any $B > 0$. For condition (ii), when $w \neq w'$, the difference $\text{diff} = H(w) - H(w')$ has coefficients that are approximately uniformly distributed over $\mathbb{Z}_q$ (by the randomness of H modeled as a random oracle), so the expected squared Euclidean distance after centered reduction is $\mathbb{E}[\text{dist}] \approx n \cdot q^2 / 12 \approx 1.29 \times 10^{10}$. Setting $B = 40$, the ratio $\mathbb{E}[\text{dist}] / B \approx 3.2 \times 10^8$ provides a safety margin of over eight orders of magnitude. Experimental verification (Section 6.6) confirms this analysis: across 10,000 non-matching keyword pairs, the minimum observed distance is 1.14×10^{10} , yielding a safety margin of $\min(\text{dist}_{\text{non-match}}) / B \approx 2.85 \times 10^8$, with zero false positives observed even when B is increased to 10^9 . The choice of $B = 40$ is therefore extremely conservative and robust across parameter variations ($\sigma \in \{2.0, 3.2, 4.0, 6.0\}$).

In summary, BL-ABSE satisfies correctness. $\square$

4.12. Complexity Analysis

This section analyzes the computational and communication complexity of each BL-ABSE algorithm and compares them with traditional constructions. Table 3 summarizes the complexity of each algorithm.

Table 3. Complexity analysis of each algorithm.

Algorithm	Computational Complexity	Communication Complexity	On-Chain Interactions	Execution Frequency
BC-Setup	$O(n^2 t)$	$O(n^2)$	3 deployments	One-time
ThresholdKeyGen	$O(t n \log n)$	$O(t n)$	$t + 1$ times	Low-frequency
BC-Encrypt	$O(n \log n)$	$O(n)$	1 time	Medium-frequency
Trapdoor	$O(n \log n)$	—	0 times	High-frequency
BC-Search	$O(\text{DB} \cdot n)$	$O(R \cdot n)$	$0 \sim 1$ times *	High-frequency
Decrypt	$O(R \cdot n \log n)$	$O(R \cdot n)$	0 times	High-frequency
RevokeAccess	$O(1)$	$O(1)$	1 time	Low-frequency
UpdatePolicy	$O(1)$	$O(1)$	1 time	Low-frequency

* Note: On-chain interactions in BC-Search are 0 (local memory queries) when caching is enabled; only 1 on-chain query is needed on a cache miss.

Key Analysis. (1) High-frequency operation efficiency: Search token generation (Trapdoor) is executed entirely locally, with no on-chain interactions, at computational complexity $O(n \log n)$. The main overhead of search matching (BC-Search) is the commitment matching traversal of the database, requiring $O(n)$ comparisons per record. With caching enabled, on-chain permission checks degrade to local memory access. (2) Low-frequency operation overhead: Key generation (ThresholdKeyGen) involves t on-chain interactions at computational complexity $O(t n \log n)$; policy registration (the on-chain portion of BC-Encrypt) requires 1 on-chain transaction. These operations are infrequent, and the additional overhead is acceptable. (3) Permission management overhead: Revocation

and policy updates only involve on-chain state modifications, with $O(1)$ computational and communication complexity, requiring no re-encryption operations.

Compared with traditional lattice-based ABSE constructions without the blockchain, the high-frequency operations (search and decryption) in BL-ABSE maintain the same asymptotic complexity, and the additional overhead introduced by the blockchain is concentrated in low-frequency management operations. This validates the core design principle of the framework—keeping high-frequency operations lightweight.

5. Security Analysis

This chapter provides rigorous formal proofs of the security properties of the BL-ABSE framework. First, we clearly delineate the scope of the formal proofs and the composition of security guarantees across system components.

5.1. Proof Scope and Composition

The BL-ABSE system comprises both cryptographic primitives and system-level infrastructure. Table 4 clarifies which components are covered by formal security proofs and which rely on standard security assumptions of their respective primitives.

Table 4. Security proof scope and composition.

Component	Security Property	Guarantee Source	Formally Proved
RLWE commitment	Index indistinguishability	Theorem 3 ($\rightarrow$ RLWE)	✓
Search token	Query privacy	Theorem 4 ($\rightarrow$ RLWE)	✓
Threshold key mgmt.	Key unrecoverability	Theorem 5 ($\rightarrow$ SSS + RLWE)	✓
PBFT consensus	Byzantine fault tolerance	Theorem 6 ($\rightarrow$ PBFT)	✓
Audit log	Tamper-evidence	Theorem 8 ($\rightarrow$ blockchain)	✓
Revocation	Immediacy	Theorem 9 ($\rightarrow$ PBFT liveness)	✓
AES-256-GCM	Data confidentiality + integrity	IND-CCA2 standard assumption	✗
IPFS	Data integrity	Content-addressed hashing	✗
Digital signature	Token unforgeability	EUF-CMA standard assumption	✗
Fabric SDK/TLS	Communication security	TLS standard assumption	✗
Attribute Authority	Attribute correctness	Trust assumption (Assumption 5)	✗

Components marked ✓ are covered by the formal proofs in this section; their security is rigorously reduced to the hardness of RLWE, SIS, or the guarantees of the PBFT consensus protocol. Components marked ✗ are treated as secure primitives under their standard cryptographic or system-level assumptions. The composition of these components follows the modular design principle: the cryptographic layer (RLWE commitments, ABE, threshold key sharing) operates independently of the blockchain enhancement layer (PBFT, smart contracts, audit logs), and the security of each layer does not depend on the internals of the other. Specifically, the IND-INDEX and IND-TRAPDOOR proofs hold regardless of the blockchain implementation, and the PBFT fault-tolerance guarantees hold regardless of the cryptographic construction.

We now proceed to prove each formal security property.

5.2. Index Indistinguishability

Index indistinguishability is a core security property of searchable encryption protocols, requiring that an adversary cannot distinguish between two different keywords from an encrypted index. The index indistinguishability of BL-ABSE reduces to the hardness of the RLWE decision problem.

Theorem 3 (IND-INDEX Security). BL-ABSE satisfies the $\mathcal{L}$ -IND-INDEX security defined in Definition 14. That is, for any PPT (or QPT) adversary $\mathcal{A}$, its advantage in

the index indistinguishability game is negligible. Specifically, there exists a PPT (or QPT) algorithm $\mathcal{B}$ such that:

$$\text{Adv}^{\text{IND-INDEX}}(\mathcal{A}) \leq \text{Adv}^{\text{RLWE}}(\mathcal{B}) + \text{negl}(\lambda). \quad (14)$$

Proof Sketch.

We employ the hybrid game reduction technique, constructing a series of hybrid games that gradually reduce the IND-INDEX game to the RLWE decision problem. The key observation is that in the keyword commitment $\text{commit} = a \cdot s + H(w)$, since s is deterministically derived by the public hash function $H_{\text{seed}}(w)$, the output of H_{seed} is computationally indistinguishable from a uniformly random string. Therefore, even though s is generated deterministically, $a \cdot s$ remains indistinguishable from a uniformly random polynomial under the RLWE assumption, so the commitment does not leak information about $H(w)$.

Proof. We define the following sequence of hybrid games:

Game 0 (Real Game): This is the standard IND-INDEX game from Definition 14. The challenger executes BC-Setup to generate public parameters pp , and the adversary $\mathcal{A}$ may adaptively issue encryption queries and search token queries. Finally, $\mathcal{A}$ submits a challenge keyword pair (w_0^*, w_1^*) and an access policy P^* , the challenger selects $b \leftarrow \{0, 1\}$ at random and returns the encrypted index I^* for w_b^* . The adversary outputs a guess b' . Define the adversary's advantage in Game 0 as:

$$\text{Adv}_0(\mathcal{A}) = |\Pr[b' = b] - 1/2|. \quad (15)$$

Game 1 (Hash Function Replacement): Replace the hash function $H_{\text{seed}} : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ with a truly random function RF . That is, for each new keyword w , $\text{RF}(w)$ returns a fresh uniformly random 256-bit string. Since H_{seed} is modeled as a random oracle (RO) in BL-ABSE, this replacement is equivalent in the random oracle model. In the standard model, if H_{seed} is instantiated as a cryptographic hash function (e.g., SHA-256), then $|\text{Adv}_0(\mathcal{A}) - \text{Adv}_1(\mathcal{A})|$ is bounded by the pseudorandomness of H_{seed} and is negligible.

Game 2 (Gaussian Sampling Replacement): Replace the deterministic Gaussian sampling derived from $\text{seed}_c = \text{RF}(w)$ with a genuinely random Gaussian sample $s \leftarrow \chi^n$. Specifically, instead of using a pseudorandom number generator to derive the coefficients of s from seed_c , each coefficient is independently sampled from the discrete Gaussian distribution $\mathcal{D}_{\mathbb{Z}, \sigma}$. Since seed_c is a uniformly random 256-bit string in Game 1, the sampling sequence expanded from seed_c by a cryptographically secure pseudorandom generator (PRG) is computationally indistinguishable from a truly random sequence. Therefore,

$$|\text{Adv}_1(\mathcal{A}) - \text{Adv}_2(\mathcal{A})| \leq \text{Adv}^{\text{PRG}}(\mathcal{B}') \leq \text{negl}(\lambda), \quad (16)$$

where $\mathcal{B}'$ is the reduction algorithm attacking the PRG and Adv^{PRG} denotes the PRG security advantage.

Game 3 (RLWE Sample Replacement): Replace the commitment in the challenge index from $\text{commit}^* = a \cdot s^* + H(w_b^*)$ to $\text{commit}^* = u + H(w_b^*)$, where $u \leftarrow R_q$ is a uniformly random polynomial. This replacement directly corresponds to the RLWE decision problem: under the RLWE assumption, $(a, a \cdot s^*)$ and (a, u) are computationally indistinguishable, where $a \leftarrow R_q$ and $s^* \leftarrow \chi^n$. Therefore,

$$|\text{Adv}_2(\mathcal{A}) - \text{Adv}_3(\mathcal{A})| \leq \text{Adv}^{\text{RLWE}}(\mathcal{B}) \leq \text{negl}(\lambda), \quad (17)$$

where $\mathcal{B}$ is the reduction algorithm attacking the RLWE decision problem.

Game 4 (Information-Theoretic Indistinguishability): In Game 3, the challenge commitment is $\text{commit}^* = u + H(w_b^*)$, where u is uniformly random. Note that the distribution of commit^* is completely independent of the choice of b : regardless of whether $b = 0$ or $b = 1$, commit^* follows a distribution of the form $u + H(w)$, where u is uniformly random. Since addition in R_q forms a group, $u + H(w_0^*)$ and $u + H(w_1^*)$ have the same uniform distribution (even if $H(w_0^*) \neq H(w_1^*)$). This means the challenge commitment leaks no information about b in the information-theoretic sense, and the adversary's advantage is strictly 0:

$$\text{Adv}_3(\mathcal{A}) = 0. \quad (18)$$

Combining the above hybrid games via the triangle inequality:

$$\text{Adv}^{\text{IND-INDEX}}(\mathcal{A}) = \text{Adv}_0(\mathcal{A}) \leq \text{Adv}^{\text{PRG}}(\mathcal{B}') + \text{Adv}^{\text{RLWE}}(\mathcal{B}) \leq \text{negl}(\lambda). \quad (19)$$

This completes the proof of Theorem 3. $\square$

Discussion. The key insight of this proof is demonstrating the security of the deterministic commitment mechanism. Although the secret polynomial s is deterministically derived from the keyword w via the public hash function H_{seed} (rather than from a user's private key), the commitment $a \cdot s + H(w)$ still does not leak information about $H(w)$. This is because: (1) the output of H_{seed} is computationally pseudorandom; (2) Gaussian samples derived from a pseudorandom seed are computationally indistinguishable from truly random Gaussian samples; and (3) under the RLWE assumption, $a \cdot s$ is indistinguishable from a uniformly random polynomial. Therefore, even though the adversary knows that H_{seed} is a public parameter, it cannot extract keyword information from the commitment.

5.3. Query Privacy

Query privacy guarantees that search tokens do not reveal information about the query keyword. The search token $T_w = (\text{commit}_q, S, ts, \text{expiry}, \sigma_{\text{user}})$ in BL-ABSE consists of a keyword commitment, a user attribute set, a timestamp, and a signature. We prove that an adversary cannot distinguish search tokens for different query keywords from commit_q .

Theorem 4 (IND-TRAPDOOR Security). *BL-ABSE satisfies the $\mathcal{L}$ -IND-TRAPDOOR security defined in Definition 15. That is, for any PPT (or QPT) adversary $\mathcal{A}$, its advantage in distinguishing the search tokens of two different keywords is negligible.*

Proof. The proof of query privacy follows a structure analogous to that of index indistinguishability. The query commitment $\text{commit}_q = a \cdot s + H(w)$ in the search token has the same structure as the commitment in the encrypted index. By the proof of Theorem 3, commit_q is computationally indistinguishable from a uniformly random polynomial, and therefore does not leak information about the keyword w .

Formally, suppose there exists an adversary $\mathcal{A}$ that can distinguish the search tokens for keywords w_0 and w_1 with non-negligible advantage. We construct a reduction algorithm $\mathcal{B}$ to attack the RLWE decision problem: $\mathcal{B}$ receives an RLWE challenge (a, b) (where $b = a \cdot s + e$ or $b \leftarrow R_q$), generates public parameters pp (including a), and returns $b + H(w_b)$ (where $b \leftarrow \{0, 1\}$) to $\mathcal{A}$ as the query commitment. If $\mathcal{A}$ can distinguish the tokens for the two keywords, then $\mathcal{B}$ can distinguish RLWE samples, contradicting the RLWE assumption.

Furthermore, the attribute set S , timestamp ts , and signature σ_{user} in the search token are public or publicly verifiable information and do not involve keyword content. The role of the timestamp and signature is to prevent token replay and forgery, not to conceal keywords. Therefore, the query privacy of the search token is entirely guaranteed by the indistinguishability of the commitment. $\square$

5.4. Threshold Key Security

BL-ABSE employs (t, n) threshold secret sharing for decentralized key management. The master key msk is split into n shares, each held by an edge node; any t nodes can collaboratively recover the complete key, while any fewer than t nodes obtain no information about msk whatsoever. This section proves the security of threshold key management.

Theorem 5 (Threshold Key Security). *Under the (t, n) threshold configuration of BL-ABSE, for any PPT adversary $\mathcal{A}$ that compromises at most $t - 1$ edge nodes, its advantage in recovering the complete master key msk is negligible.*

Proof. BL-ABSE employs Shamir's secret sharing protocol, whose information-theoretic security guarantees that any $t - 1$ or fewer shares reveal no information about the secret msk in the information-theoretic sense.

Specifically, during the system initialization phase (Algorithm 1), each node EN_i generates a random degree- $(t - 1)$ polynomial $f_i(x) = \text{msk}_i + a_{i,1} \cdot x + \dots + a_{i,t-1} \cdot x^{t-1} \bmod q$, where msk_i is the secret contribution of that node and $a_{i,1}, \dots, a_{i,t-1} \leftarrow \mathbb{Z}_q$ are random coefficients. Node EN_i computes the share $s_{i,j} = f_i(j)$ and sends it secretly to node EN_j . Finally, node EN_j holds the complete share:

$$\text{sk}_j = \sum_{i=1}^n s_{i,j} = \sum_{i=1}^n f_i(j) = f(j), \quad (20)$$

where $f(x) = \sum_i f_i(x)$ is the sum of all nodes' polynomials, still a degree- $(t - 1)$ polynomial, whose constant term $f(0) = \sum_i \text{msk}_i = \text{msk}$ is the master key.

By the information-theoretic security of Shamir's secret sharing, any $k < t$ shares $\{\text{sk}_j\}_{j \in T}$, $|T| = k$, correspond to k point-value constraints on $f(x)$. Since $f(x)$ is a degree- $(t - 1)$ polynomial with t degrees of freedom (t coefficients), $k < t$ constraints cannot uniquely determine $f(x)$. More precisely, for any candidate master key msk' , there exists at least one degree- $(t - 1)$ polynomial $f'(x)$ consistent with the known k shares such that $f'(0) = \text{msk}'$. This means that from the perspective of k shares, all possible master key values are equally probable, and the adversary gains zero information about msk .

Therefore, an adversary that compromises at most $t - 1$ nodes cannot obtain any information about the master key msk , let alone recover it. Theorem 5 is proved. $\square$

Corollary 1. *In a typical $(3, 5)$ configuration, an adversary that compromises 2 nodes cannot recover the master key. In a $(4, 7)$ configuration, an adversary that compromises 3 nodes cannot recover the master key.*

Lemma 1 (PBFT-Threshold Parameter Constraint). *For BL-ABSE to simultaneously satisfy both PBFT consensus security and threshold key security, the threshold parameter t must satisfy $t > f_{\text{PBFT}} + 1$, where $f_{\text{PBFT}} = \lfloor (n - 1)/3 \rfloor$ is the maximum number of Byzantine nodes tolerated by PBFT.*

Proof. The PBFT consensus mechanism tolerates at most f_{PBFT} malicious nodes while guaranteeing safety and liveness. Meanwhile, the (t, n) Shamir secret sharing scheme ensures information-theoretic security as long as the adversary controls fewer than t nodes. For both guarantees to hold simultaneously, we require: (1) the adversary cannot reach the threshold t even when controlling the maximum number of PBFT-tolerable malicious nodes, i.e., $f_{\text{PBFT}} < t$; (2) there must exist enough honest nodes to both reach PBFT consensus and participate in key generation, i.e., $n - f_{\text{PBFT}} \geq t$.

If $t \leq f_{\text{PBFT}}$, an adversary controlling exactly f_{PBFT} nodes (which PBFT considers tolerable) would hold t or more shares, enabling full master key reconstruction—a catastrophic breach. Setting $t > f_{\text{PBFT}} + 1$ ensures a safety margin of at least 2 between the PBFT tolerance bound and the reconstruction threshold. Furthermore, since $n - f_{\text{PBFT}} \geq 2f_{\text{PBFT}} + 1 > t$ for typical configurations, enough honest nodes exist to complete key generation. Table 5 shows recommended parameter configurations. $\square$

Table 5. Recommended PBFT-threshold parameter configurations.

n (Nodes)	f_{PBFT}	Recommended t	Safety Margin $t - f_{\text{PBFT}}$	Notes
4	1	3	2	Minimum practical config.
5	1	3	2	Default config. in this paper
7	2	4	2	Standard deployment
10	3	5	2	Large-scale deployment
13	4	7	3	High-security deployment

5.5. Byzantine Fault Tolerance

BL-ABSE is built on the PBFT consensus mechanism of Hyperledger Fabric, ensuring system security and liveness even when some edge nodes are malicious or faulty. This section analyzes the Byzantine fault tolerance properties of the framework.

Theorem 6 (Byzantine Fault Tolerance Security). *BL-ABSE satisfies the Byzantine fault tolerance property defined in Definition 17. When the total number of edge nodes $n \geq 3f + 1$, the system can tolerate at most $f = \lfloor (n - 1)/3 \rfloor$ Byzantine nodes while guaranteeing: (1) Safety: honest nodes will not accept invalid key shares or blockchain transactions; (2) Liveness: legitimate users' key requests and search requests will eventually be processed.*

Proof. The correctness of the PBFT consensus protocol has been proved in classical literature. We explain how BL-ABSE leverages PBFT to achieve Byzantine fault tolerance, and provide specific safety and liveness arguments.

(1) Safety requires that honest nodes will not accept invalid data submitted by malicious nodes. BL-ABSE guarantees safety through the following mechanisms:

Zero-Knowledge Proof Verification: During threshold key generation (Algorithm 2), each node must submit a zero-knowledge proof π_i together with its partial key $sk^{(i)}$, proving that $sk^{(i)}$ was correctly computed from that node's share sk_i . The KeyManagement smart contract verifies all proofs before executing key aggregation. Even if f malicious nodes submit incorrect partial keys, as long as at least $2f + 1$ honest nodes execute correctly and the contract requires at least t valid proofs ($t \geq 2f + 1$), the aggregation result remains correct.

PBFT Consensus Verification: All transactions on the blockchain (policy registration, permission revocation, audit logs, etc.) are confirmed through PBFT consensus. PBFT guarantees that under $n \geq 3f + 1$, even with f malicious nodes, all honest nodes reach agreement on the validity and execution order of transactions. Specifically, the three-phase protocol of PBFT (Pre-Prepare, Prepare, Commit) ensures that a transaction is confirmed if and only if at least $2f + 1$ nodes approve it, and with at most f malicious nodes, at least $f + 1$ honest nodes participate in confirmation, guaranteeing transaction validity.

(2) Liveness requires that legitimate users' requests will eventually be processed. The liveness of BL-ABSE depends on the following conditions:

Sufficient Honest Nodes: As long as the number of honest nodes $h \geq 2f + 1$, the system can reach consensus. In threshold key generation, the parameter t is typically set to $\lceil (n + 1)/2 \rceil$ or larger, ensuring that a majority of nodes must participate. Under the

condition $n \geq 3f + 1$, the number of honest nodes $h \geq 2f + 1 > t$, so there are always enough honest nodes to complete key generation.

Timeout and Retry Mechanism: If some nodes refuse to participate or have delayed responses, the protocol design of BL-ABSE includes a timeout mechanism. In key generation, if the waiting time exceeds a preset threshold, the system may select other honest nodes to re-initiate the protocol. Since the number of honest nodes is sufficient, t responding honest nodes can always eventually be found.

Liveness Guarantee of PBFT: The PBFT protocol guarantees liveness under the network synchrony assumption—even in the presence of f malicious nodes and network delays, once the network eventually recovers synchrony, legitimate transactions will be confirmed. The Hyperledger Fabric implementation further handles primary node failures through the View Change mechanism, ensuring continuous system operation.

Combining the above analysis, BL-ABSE satisfies both safety and liveness under the configuration $n \geq 3f + 1$. Theorem 6 is proved. $\square$

5.6. Post-Quantum Security

The core cryptographic constructions of BL-ABSE are based on the RLWE and SIS problems, neither of which currently has a known polynomial-time quantum algorithm. Consequently, the framework achieves post-quantum security. This section discusses security in the quantum adversary model.

Theorem 7 (Post-Quantum Security). *In Theorems 3–5, if PPT adversaries are replaced by QPT (quantum polynomial-time) adversaries, the security conclusions of BL-ABSE still hold. That is, BL-ABSE satisfies post-quantum IND-INDEX security, post-quantum IND-TRAPDOOR security, and post-quantum threshold key security.*

Proof Sketch. The proof of post-quantum security relies on the resistance of lattice problems against quantum algorithms. We analyze each aspect:

(1) **Quantum Hardness of the RLWE Problem:** The hardness of the RLWE decision problem under the quantum computing model has been extensively studied. In his seminal 2005 work, Regev proved that the standard LWE problem admits worst-case reductions from the GapSVP and SIVP problems on lattices. Lyubashevsky et al. extended this result to RLWE, proving that the hardness of RLWE in the quantum computing model can be reduced to approximation variants of the shortest vector problem on ideal lattices.

The best currently known quantum algorithms for approximating SVP on an n -dimensional lattice still require $2^{\Omega(n)}$ time. Even accounting for the quadratic speedup provided by Grover's algorithm (reducing classical brute-force search from $O(2^n)$ to $O(2^{n/2})$), for the parameter $n = 1024$ used in BL-ABSE, the complexity of quantum attacks remains 2^{512} , far beyond feasible computation. Therefore, the RLWE problem remains hard in the quantum computing model.

(2) **Quantum Hardness of the SIS Problem:** The binding property of the BL-ABSE commitment construction reduces to the Short Integer Solution (SIS) problem. SIS requires finding a short vector $\mathbf{x}$ such that $\mathbf{Ax} = 0 \bmod q$, where $\mathbf{A}$ is a random matrix. The SIS problem can be reduced to SVP on lattices, thus inheriting the quantum hardness of SVP. The survey by Micciancio and Regev indicates that the optimal quantum algorithms for the SIS problem still require exponential time.

(3) **Information-Theoretic Security of Threshold Secret Sharing:** The security of Shamir's secret sharing protocol is information-theoretic and does not rely on any computational assumption. Even if the adversary has unlimited quantum computing power, any $t - 1$ or fewer shares still reveal no information about the secret in the information-

theoretic sense. Therefore, the security of threshold key management is fully preserved in the quantum model.

(4) **Quantum Security of Symmetric Cryptography:** BL-ABSE uses AES-256-GCM for data encryption. Although Grover's algorithm can reduce the security of AES-256 from 256 bits to 128 bits (through quadratic-speedup exhaustive search), a 128-bit security level is still considered sufficient in practice. For higher assurance, one may upgrade to AES-512 (if standardized) or adopt other quantum-resistant symmetric ciphers such as ChaCha20.

Combining the above analysis, the core security properties of BL-ABSE continue to hold in the quantum computing model. Theorem 7 is proved. $\square$

NIST Post-Quantum Cryptography Standardization Validation. The lattice-based cryptography adopted by BL-ABSE has been selected in the NIST Post-Quantum Cryptography Standardization process. In 2022, NIST announced the first batch of post-quantum cryptographic standards, in which CRYSTALS-Kyber (based on Module-LWE) was selected as the key encapsulation mechanism standard and CRYSTALS-Dilithium (based on Module-SIS) was selected as the digital signature standard. These constructions share the same RLWE/SIS assumptions as BL-ABSE, further validating the quantum-resistance of lattice-based cryptography.

5.7. Audit Tamper-Evidence

BL-ABSE stores audit logs on the blockchain, leveraging the immutability of the blockchain to ensure traceability and integrity of the operation history. This section analyzes the security properties of audit logs.

Theorem 8 (Audit Tamper-Evidence). *BL-ABSE satisfies the audit tamper-evidence property defined in Definition 18. That is, for any PPT adversary $\mathcal{A}$, the probability of modifying historical audit logs without being detected is negligible.*

Proof. The immutability of the blockchain stems from the combination of its cryptographic hash-chain structure and consensus mechanism. We analyze how BL-ABSE leverages these properties to guarantee the integrity of audit logs.

(1) **Cryptographic Hash Chain:** Each block in the blockchain contains the cryptographic hash (e.g., SHA-256) of the preceding block. To modify an audit log in block i , an adversary must: (i) modify the content of that block; (ii) recompute the hash of that block; (iii) modify the predecessor hash stored in block $i + 1$; (iv) recursively modify all subsequent blocks.

Since SHA-256 satisfies collision resistance (the probability of finding two distinct inputs that produce the same hash is negligible), the adversary cannot modify block content without changing the hash value. Therefore, any modification to a historical block will break the hash chain and be detected by honest nodes.

(2) **Distributed Consensus Verification:** Even if an adversary can modify the blockchain replica on its own node, the modified chain is inconsistent with the chains of other honest nodes. Under the PBFT consensus mechanism, confirming a block requires the approval of at least $2f + 1$ nodes. For the adversary to have its modified block accepted by the system, it would need to control at least $2f + 1$ nodes, which requires $f \geq (n + 1)/3$, violating the assumption $n \geq 3f + 1$.

Furthermore, even if the adversary controls $f < (n + 1)/3$ nodes and attempts to fork the chain (creating a parallel chain with falsified logs), the finality guarantee of PBFT ensures that confirmed blocks will not be rolled back. Honest nodes will refuse to accept any chain inconsistent with the confirmed history.

(3) **Multi-Party Witnessing:** Audit logs in BL-ABSE are recorded by the AuditLog smart contract, with each recording operation corresponding to a blockchain transaction. Once a transaction is confirmed, its hash and timestamp are permanently written on-chain and witnessed by all n nodes. To tamper with an audit log entry, the adversary would need to simultaneously control more than $2/3$ of the nodes, which is impossible under the honest-majority assumption.

Combining the above analysis, under the collision-resistance assumption of SHA-256 and the security guarantees of PBFT, an adversary cannot modify historical audit logs without being detected. Theorem 8 is proved. $\square$

Practical Detection Mechanisms. BL-ABSE deploys the following mechanisms to enhance the verifiability of audit logs: (1) Blockchain Explorer: provides a graphical interface for querying and verifying audit logs, displaying the transaction hash, block number, and timestamp for each log entry; (2) Merkle Proof: for scenarios that require proving the existence of a log entry to an external auditor, the system can generate a Merkle path from the log entry to the block root hash, providing a concise existence proof; (3) Periodic Snapshots: the system periodically publishes the Merkle root of the blockchain state to publicly trusted platforms (e.g., newspapers, notary services), further enhancing the verifiability of historical records.

5.8. Revocation Immediacy

BL-ABSE achieves immediate permission revocation through an on-chain revocation list. Compared with traditional approaches that require re-encryption of data (which can take hours), on-chain revocation takes effect immediately upon block confirmation (approximately 2 s). This section formalizes this property.

Theorem 9 (Revocation Immediacy). *BL-ABSE satisfies the revocation immediacy property defined in Definition 19. That is, if user u is revoked access to data d at time t , then for any search request issued at time $t' > t + \Delta_{\text{confirm}}$ (where $\Delta_{\text{confirm}} \approx 2$ s is the blockchain confirmation delay), user u cannot retrieve data d .*

Proof. The proof of revocation immediacy is divided into two parts: the time bound for revocation taking effect and the enforcement of revocation.

(1) **Revocation Effective Time:** At time t , the data owner invokes the `RevokeAccess(dataId, userId)` algorithm (Algorithm 7), which submits a revocation transaction to the blockchain. Under PBFT consensus, transaction confirmation requires: (i) broadcasting the transaction to all nodes (network delay < 100 ms); (ii) the three-phase consensus protocol (Pre-Prepare, Prepare, Commit), with each phase requiring message exchanges between nodes; (iii) writing the transaction into a new block.

In a typical Hyperledger Fabric configuration, the block generation time is set to 2 s (Algorithm 1, Line 6). Therefore, the time from submitting the revocation transaction to confirming it on the blockchain is $\Delta_{\text{confirm}} \approx 2$ s (plus network delay, in practice 2–3 s). Once the transaction is confirmed, the revocation record becomes immediately visible to all honest nodes.

(2) **Enforcement of Revocation:** The search algorithm `BC-Search` (Algorithm 5) performs an on-chain revocation check (Line 7) before returning results:

if `AccessControl.isRevoked(dataId, userId)` then continue. (21)

This check queries the latest state on the blockchain. For a search request issued at time $t' > t + \Delta_{\text{confirm}}$, the revocation transaction has already been confirmed,

$\text{AccessControl.isRevoked}(d, u)$ returns true, and the search algorithm skips the data item without adding it to the result set.

It is worth noting that the revoked user u still holds a valid user private key sk_u and search token T_w at time t' . However, since the revocation check in the search algorithm is mandatory (Algorithm 5, Line 7, executed unconditionally), even if the user attempts to directly access the cloud storage to obtain the ciphertext CID, they still cannot bypass revocation—data access is protected by ABE ciphertext, and the ABE ciphertext policy may have been updated after revocation (via UpdatePolicy); or even if the policy remains unchanged, the audit log will record the unauthorized access attempt and trigger a security alert.

Combining the above analysis, BL-ABSE guarantees that all search requests from a revoked user will be rejected within approximately 2 s after the revocation transaction is confirmed. Theorem 9 is proved. $\square$

Comparison with Traditional Approaches. Traditional attribute-based searchable encryption constructions achieve revocation through key updates or data re-encryption. Key update approaches require regenerating private keys for all non-revoked users and securely distributing them, which may take minutes; data re-encryption methods require re-encrypting all affected data and updating indexes, which may take hours or even days for large-scale datasets. In contrast, BL-ABSE's on-chain revocation list mechanism reduces revocation latency to the second level, without requiring any re-encryption or key redistribution, substantially improving the efficiency of dynamic permission management.

5.9. Search Pattern Leakage Analysis

The deterministic commitment mechanism of BL-ABSE—wherein the same keyword w always produces the same commitment $\text{commit} = a \cdot s + H(w)$ —is essential for non-interactive search matching but inherently leaks the *search pattern*: an adversary observing encrypted indices and search tokens can determine whether two queries target the same keyword. This section quantitatively analyzes three concrete attack vectors and characterizes the resulting privacy implications.

5.9.1. Theoretical Frequency Analysis

An adversary who observes a stream of search tokens can count the frequency of each distinct commitment and attempt to match these frequencies against a known keyword distribution (e.g., Zipf's law). Our experimental analysis (Section 6.8) evaluates this attack across varying vocabulary sizes ($|\mathcal{W}| \in \{100, 500, 1000\}$) and distribution skewness parameters ($\alpha \in \{1.0, 1.5, 2.0\}$). The results show that keyword recovery rates remain modest in realistic settings: for $|\mathcal{W}| = 1000$ and $\alpha = 1.0$ (near-uniform distribution), the recovery rate is only 2.5% even after observing 10,000 queries. However, for small vocabularies with highly skewed distributions ($|\mathcal{W}| = 100, \alpha = 2.0$), the rate increases to 14.9%, indicating that frequency analysis is more effective when the keyword space is small and queries are concentrated on a few popular terms.

5.9.2. Dictionary Attack

Since H_{seed} is a public parameter, an adversary with a candidate keyword dictionary $\mathcal{D}$ can precompute commitments for all $w \in \mathcal{D}$ and compare them against observed commitments. This is a more powerful attack than frequency analysis: when $\mathcal{D}$ covers the actual keyword space, the adversary achieves 100% keyword identification. This is a fundamental consequence of the deterministic design and is shared by all searchable encryption protocols that support non-interactive keyword matching with equality testing.

The primary defense is to ensure that the effective keyword space is large and that keywords are drawn from a sufficiently high-entropy distribution.

5.9.3. Cross-User Linkage

Because the commitment derivation uses only the public parameters a , H , and H_{seed} (and not user-specific keys), different users searching for the same keyword produce identical commitments. An adversary can thus link queries across users with certainty. Experimental results confirm that 100% of user pairs sharing at least one keyword are linkable, with an average of approximately 20 shared keywords per pair (out of 100 queries each from a vocabulary of 500).

5.9.4. Correctness–Privacy Trade-Off

The search-pattern leakage described above is *not* a flaw in the security proof but rather a fundamental design trade-off: deterministic commitments are necessary for non-interactive, exact-match search correctness (Theorem 3), while randomized commitments would eliminate pattern leakage at the cost of requiring interactive protocols or server-side trial decryption, significantly increasing communication complexity and search latency.

BL-ABSE explicitly operates within the $\mathcal{L}$ -IND-INDEX security framework (Definition 14), where the leakage function $\mathcal{L}_{\text{Search}}$ includes the search pattern $\text{sp}(w) = \{i : w_i = w\}$. The formal proofs in Theorems 3 and 4 guarantee that no information *beyond* this declared leakage is revealed. Known inference attacks (e.g., file-injection attacks [3], query recovery attacks [4]) exploit precisely this leakage; the defense strategies discussed in Section 7.2 (ORAM integration, differential privacy) can compress the exploitable leakage at the cost of additional overhead.

6. Experimental Evaluation

6.1. Experimental Setup

All experiments are conducted on a server equipped with an Intel Xeon E5-2680 v4 processor (Intel Corporation, Santa Clara, CA, USA) (2.4 GHz, 14 cores), 64 GB DDR4 RAM, running Ubuntu 22.04 LTS (Canonical Ltd., London, UK). The prototype system is implemented in Python 3.10 (Python Software Foundation, Wilmington, DE, USA) with NumPy 1.24 for polynomial arithmetic and hashlib/SHAKE-256 for hash functions. Blockchain experiments use Hyperledger Fabric 2.5 (Linux Foundation, San Francisco, CA, USA) with PBFT consensus configured at a 2-s block time, deployed across Docker containers (Docker Inc., Palo Alto, CA, USA). Each experimental group is independently repeated 30 times unless otherwise stated, with mean $\pm$ standard deviation reported.

The RLWE parameters are $n = 1024$, $q = 12,289$, $\sigma = 3.2$, and $B = 40$ throughout all experiments, consistent with the parameter choices in Section 4.2. The NTT implementation uses precomputed root-of-unity tables and bit-reversal permutation, as described in Section 2.3.

6.2. NTT Optimization Effectiveness Evaluation

As noted in Section 4.4, the keyword commitment generation of the BL-ABSE framework relies on polynomial ring multiplication in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. The naïve implementation has a computational complexity of $O(n^2)$, which introduces significant computational bottlenecks at the lattice dimension $n = 1024$. To this end, the framework introduces Number Theoretic Transform (NTT) optimization based on negacyclic convolution, reducing polynomial multiplication complexity to $O(n \log n)$. This section verifies the practical effectiveness of this optimization through comparative experiments.

Experiments are conducted at four sets of lattice dimensions $n \in \{256, 512, 1024, 2048\}$. The RLWE parameters are fixed at $q = 12,289$ and $\sigma = 3.2$, consistent with the parameter

choices in Section 4.2 for system initialization. For each parameter set, 50 commitment generation operations are executed separately using naïve polynomial multiplication and NTT-optimized multiplication, with average execution time and standard deviation recorded. The precomputation overhead of NTT (construction of the table of roots of unity and the bit-reversal permutation table) is completed once during system initialization and is not counted in the single commitment generation time, consistent with the actual deployment execution flow. Results are shown in Table 6.

Table 6. Comparison of keyword commitment generation time before and after NTT optimization.

Lattice Dim. n	Naive Method Avg. (ms)	Naive Method Std. (ms)	NTT Opt. Avg. (ms)	NTT Opt. Std. (ms)
256	23.38	1.53	4.93	0.50
512	103.56	14.10	10.76	0.69
1024	401.43	13.01	23.13	0.68
2048	1615.67	31.98	51.63	1.53

Note: Bold values in the $n = 1024$ row indicate the default parameter configuration adopted by BL-ABSE, which achieves 128-bit post-quantum security.

From Table 6, the following observations can be made. First, NTT optimization achieves significant performance gains across all tested dimensions. At the core parameter $n = 1024$ of this framework, the naïve method has an average commitment generation time of 401.43 ms, which is reduced to 23.13 ms after NTT optimization, yielding a speedup of 17.4×. Second, the speedup increases monotonically with the lattice dimension, growing from 4.7× at $n = 256$ to 31.3× at $n = 2048$. This trend is consistent with theoretical analysis: the complexity of the naïve method is $O(n^2)$ and that of the NTT method is $O(n \log n)$, whose ratio asymptotically approaches $O(n / \log n)$, increasing as n grows. Finally, the standard deviation of NTT-optimized execution times is significantly smaller than that of the naïve method (e.g., 0.68 ms vs. 13.01 ms at $n = 1024$), indicating that NTT execution times are more stable, which has important implications for latency predictability in edge computing scenarios.

These results validate the effectiveness of the NTT optimization design in Section 4.4, showing that BL-ABSE can control the computational latency of keyword commitment generation at the millisecond level while maintaining the 128-bit post-quantum security level ($n = 1024$), meeting the performance requirements of real-time encrypted search on edge devices. It should be noted that this experiment is based on a Python prototype implementation; engineering optimizations at the C/C++ level (such as Montgomery reduction discussed in Section 2.3) can further reduce absolute execution time.

Security–Efficiency Trade-off. Table 6 also reveals the security–efficiency trade-off inherent in the lattice dimension parameter n . As n increases from 256 to 2048, the security level grows from approximately 64-bit to 256-bit post-quantum security, while the NTT-optimized commitment generation time increases from 4.93 ms to 51.63 ms—a 10.5× increase in computation for a 4× increase in security. At the chosen parameter $n = 1024$ (128-bit security), the commitment generation time of 23.13 ms represents a practical sweet spot for edge-computing scenarios where sub-100 ms latency is required.

6.3. Comparative Analysis with Existing Approaches

To comprehensively evaluate the functional completeness and technical advantages of BL-ABSE, this section systematically compares BL-ABSE with thirteen representative constructions published in recent years, across two dimensions: functional features and computational complexity. The selected works cover three main research directions: attribute-based searchable encryption (ABSE), post-quantum searchable encryption, and blockchain-enhanced searchable encryption.

6.3.1. Comparison of Functional Features

Table 7 compares the functional features of each approach across nine key dimensions. The results show that BL-ABSE is the only framework that simultaneously satisfies all functional requirements.

On the post-quantum security dimension, constructions based on bilinear pairings [9–11,26,27] all rely on discrete logarithm or DBDH assumptions and will be completely broken against quantum computing attacks. Shen et al. [22], Cao et al. [24], Huang et al. [25], Yang et al. [23], Liu et al. [21], and Zhang et al. [20] introduce lattice-based constructions to achieve post-quantum security, but none simultaneously addresses a decentralized trust model and on-chain governance mechanisms.

On the decentralized key management dimension, Yu et al. [26] and Shen et al. [22] achieve partial decentralization through a multi-authority architecture, but the former is based on traditional cryptographic assumptions while the latter does not incorporate blockchain enhancement. Yin et al. [27] achieve threshold key management through the blockchain but lack post-quantum security guarantees. The remaining works [9,11,20,21,23–25,28] all rely on a single key generation center, with single-point-of-failure and key escrow risks.

On the permission revocation dimension, approaches differ significantly. In terms of revocation granularity, [22,24,25,28] support only attribute-level revocation, Ref. [30] supports only user-level revocation, [24] supports both granularities but relies on a centralized key generation center, while BL-ABSE implements both attribute-level and user-level revocation simultaneously through smart contracts without requiring a trusted third party.

Regarding audit traceability, only Yin et al. [27] and Yan et al. [30] provide limited audit capabilities using the blockchain, but the audit granularity is limited and does not cover key operations such as search queries and policy changes. BL-ABSE leverages the immutability of the blockchain to record complete operation logs, providing cryptographic-level evidence for post-hoc accountability.

Table 7. Comparison of functional features between BL-ABSE and existing approaches.

Approach	Security Assumption	PQ Secure	Access Control	Decen. Key Mgmt	Blockchain	Instant Revoc.	Audit Log	NTT Opt.
Zhang et al. [10]	DBDH/DL	✗	CP-ABE	✗	✗	✗	✗	✗
Yu et al. [26]	DBDH/DL	✗	MA-ABE	△	✓	✓	✗	✗
Luo et al. [9]	DBDH/DL	✗	CP-ABE	✗	✗	✗	✗	✗
Niu et al. [11]	DBDH/DL	✗	CP-ABE	✗	✗	△	✗	✗
Yin et al. [27]	DBDH/DL	✗	CP-ABE	✓	✓	✗	△	✗
Shen et al. [22]	LWE	✓	MA-ABE	△	✗	✓	✗	✗
Cao et al. [24]	LWE	✓	CP-ABE	✗	✗	✗	✗	✗
Huang et al. [25]	LWE	✓	ABE	✗	✗	✗	✗	✗
Yang et al. [23]	RLWE	✓	CP-ABE	✗	✗	✗	✗	△
Liu et al. [21]	RLWE/SIS	✓	PEKS	✗	✗	✗	✗	✗
Zhang et al. [20]	LWE	✓	PEKS	✗	✗	✗	✗	✗
Wang et al. [28]	q-BDHE/DDH	✗	CP-ABE	✗	✗	✗	✗	✗
Yan et al. [30]	DBDH/DL	✗	ABE	✗	✓	✗ ^a	✓	✗
BL-ABSE (Ours)	RLWE/SIS	✓	CP-ABE	✓	✓	✓	✓	✓

Note: ✓ = fully supported; △ = partially supported; ✗ = not supported. Bold values in the last row indicate the proposed BL-ABSE approach. ^a Yan et al. [30] use the blockchain but the revocation process still requires proxy re-encryption.

6.3.2. Comparison of Computational Complexity

Table 8 provides a quantitative comparison of representative approaches on the computational complexity dimension. In the encryption and token generation phases, the core overhead of bilinear-pairing-based constructions [10,26] is $O(|U|)$ or $O(|S|)$ pairing operations, which becomes significant for large attribute sets. Constructions based on naïve lattice operations [21,22], while achieving post-quantum security, have $O(n^2)$ polynomial multiplication complexity, resulting in high per-encryption latency at $n = 1024$ that is difficult to meet in real-time deployment scenarios. Yang et al. [23] partially introduce NTT optimization through an offline/online strategy but do not provide a complete NTT-accelerated implementation and only optimize the encryption phase. BL-ABSE, through

complete NTT optimization based on negacyclic convolution, reduces polynomial multiplication from $O(n^2)$ to $O(n \log n)$, and achieves computational acceleration in all three phases of encryption, token generation, and search by combining precomputed tables of roots of unity, bit-reversal permutations, and Montgomery reduction.

Table 8. Comparison of computational complexity between BL-ABSE and existing approaches.

Approach	Encryption Complexity	Search Complexity	Token Gen. Complexity	Key Storage
Zhang et al. [10]	$O(U)P$	$O(DB)P$	$O(S)P$	$O(S)$
Yu et al. [26]	$O(U)P$	$O(DB)P$	$O(S)P$	$O(S)$
Shen et al. [22]	$O(n^2)$	$O(DB \cdot n^2)$	$O(n^2)$	$O(n)$
Yang et al. [23]	$O(n \log n)$	$O(DB \cdot n)$	$O(n \log n)$	$O(n)$
Liu et al. [21]	$O(n^2)$	$O(DB \cdot n^2)$	$O(n^2)$	$O(n)$
BL-ABSE (Ours)	$O(n \log n)$	$O(DB \cdot n)$	$O(n \log n)$	$O(n)$

Note: P denotes one bilinear pairing operation; n is the lattice dimension ($n = 1024$ in this work); $|DB|$ is the database size; $|U|$ is the size of the attribute universe; $|S|$ is the size of the user attribute set. Bold values in the last row indicate the proposed BL-ABSE approach.

The performance disadvantage of traditional approaches is particularly prominent in the permission revocation dimension. The revocation mechanism of Shen et al. [22] requires regenerating keys for all non-revoked users, with time complexity $O(|U_r|) \cdot \text{KeyUpd}$; when the system is large, revocation latency can reach the hour level. Yu et al. [26], with blockchain assistance for revocation, still require $O(|DB|)$ bilinear pairing operations for data re-encryption. BL-ABSE reduces the time complexity of permission revocation to $O(1)$ using an on-chain revocation list mechanism—the revocation operation requires only one blockchain transaction (approximately 2 s for confirmation), without any data re-encryption or key redistribution. This advantage is particularly important in large-scale systems requiring frequent permission changes.

Combining the above analysis, BL-ABSE demonstrates significant advantages in both functional completeness and computational efficiency. On the functional level, BL-ABSE is the first to organically integrate lattice-based post-quantum security, attribute-based fine-grained access control, blockchain decentralized trust, immediate permission revocation, and tamper-evident audit logs into a unified framework. Its core design philosophy lies in migrating revocation state management from traditional cryptographic mechanisms (key updates or data re-encryption) to blockchain smart contracts, decoupling the cost of revocation from system scale. On the efficiency level, BL-ABSE controls the computational complexity of high-frequency search operations to $O(n \log n)$ through NTT optimization, while concentrating blockchain interactions on low-frequency management operations, achieving an effective balance between security and practicability.

6.4. Relationship Between Search Latency and Keyword Scale

Search latency is one of the most critical performance metrics for searchable encryption protocols in practical deployments, directly determining user query experience and system service quality. The BC-Search algorithm (Algorithm 5) of BL-ABSE needs to traverse all index records in the encrypted database and perform RLWE commitment matching operations on each record; therefore, its search latency should theoretically increase linearly with the number of keyword indexes in the database. This experiment aims to validate this theoretical expectation through systematic empirical measurements, evaluating the search performance of the framework at different data scales, and providing quantitative references for capacity planning in actual deployments.

Experimental Setup

The experiment uses the RLWE parameter configuration consistent with the protocol design: lattice dimension $n = 1024$, prime modulus $q = 12,289$ (satisfying $q \equiv 1 \pmod{2n}$ to support NTT), discrete Gaussian standard deviation $\sigma = 3.2$, and commitment matching threshold $B = 40$. This parameter combination guarantees a 128-bit post-quantum security level, comparable to the parameter choices of the CRYSTALS series constructions in the NIST post-quantum cryptography standardization process.

The experiment constructs an encrypted database with 100 to 50,000 keyword indexes, with 8 specific test scale gradients: 100, 500, 1000, 2000, 5000, 10,000, 20,000, and 50,000. Each keyword index is generated following the procedure of Algorithm 3 (BC-Encrypt): first, the public hash function H_{seed} is used to deterministically derive the Gaussian sampling seed from the keyword, and then the RLWE commitment is computed via NTT-optimized polynomial multiplication. The search phase executes the complete BC-Search algorithm, including triple verification of token signature verification, on-chain policy and revocation status checking (in local cache mode), and RLWE commitment matching. To eliminate random fluctuations, each set of experiments is independently repeated 30 times; the average is taken as the final result, with minimum and maximum values recorded to assess result stability.

Results and Analysis

Figure 3 shows the experimental results of search latency varying with the number of keywords. The light blue shaded area represents the range from minimum to maximum search latency across 30 repeated experiments, and the red dashed line is the linear fit based on the empirical data.

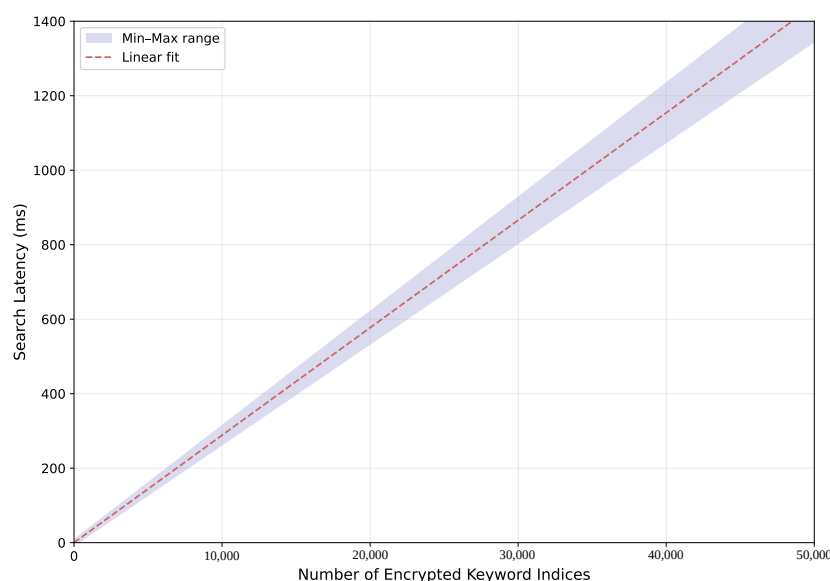


Figure 3. Relationship between search latency and number of keywords.

The experimental results show that search latency and the number of keywords exhibit a highly linear growth relationship. When the database contains 100 keyword indexes, the average search latency is only 2.79 ms; as the number of keywords grows to 1000, 10,000, and 50,000, the average search latency increases to 29.29 ms, 290.32 ms, and 1443.79 ms, respectively. Table 9 provides detailed measurement data for each scale gradient.

The linear fit results in Figure 3 show that the fitted line matches the empirical data points closely, with a fitted slope of 28.86 $\mu\text{s}/\text{keyword}$, indicating that each additional keyword index record increases search latency linearly by 28.86 μs . This result empirically validates the theoretical computational complexity $O(|\text{DB}| \cdot n)$ of the BC-Search algorithm—search latency is strictly proportional to database size. Further analysis of per-record matching time in Table 9 shows that dividing the total search latency by the number of keywords for each group yields a stable average matching time per record of 27.9–29.7 μs , which does not vary with database size. This indicates that the single execution overhead of RLWE commitment matching operations—including polynomial coefficient difference computation, centered modular reduction, and Euclidean distance computation—is constant; the growth in search latency is entirely attributable to the increasing number of index records that need to be traversed.

Table 9. Search latency experimental data.

Keywords	Avg. (ms)	Std. (ms)	Min (ms)	Max (ms)	Per-Record (μs)
100	2.79	0.05	2.72	3.05	27.9
500	14.87	0.68	13.97	17.25	29.7
1000	29.29	1.03	28.23	32.62	29.3
2000	58.01	2.21	55.83	65.57	29.0
5000	146.53	5.70	139.35	163.34	29.3
10,000	290.32	9.17	280.22	325.56	29.0
20,000	576.35	19.78	544.40	631.12	28.8
50,000	1443.79	26.00	1361.24	1495.07	28.9

From the perspective of result stability, the Min-Max shaded region in Figure 3 closely encloses the fitted line throughout, with limited variation between minimum and maximum values across experimental groups. Taking the 50,000-keyword scale as an example, the minimum value across 30 repeated experiments is 1361.24 ms, the maximum is 1495.07 ms, and the standard deviation is only 26.00 ms, with a coefficient of variation of approximately 1.8%. Smaller scale gradients exhibit even better stability; for example, at 100 keywords, the standard deviation is only 0.05 ms. This stability characteristic indicates that BL-ABSE's search performance has good predictability, facilitating quality-of-service guarantees and resource scheduling in practical systems.

6.5. Permission Revocation Efficiency Comparison Experiment

Permission revocation latency is a core metric for evaluating the practicality of dynamic access control mechanisms, directly determining the security window during which a revoked user can continue to access sensitive data. This section quantitatively evaluates the revocation time overhead of BL-ABSE and four representative baseline approaches at different user scales through comparative experiments.

The experiment adopts a simulation modeling method, constructing performance models based on algorithm descriptions and empirical data from the published papers of each baseline approach. Baseline operation time values are derived from literature measurements in a typical server environment (2.4 GHz processor): one bilinear pairing operation takes 4.2 ms, lattice polynomial multiplication (naïve implementation) takes 401.4 ms, and Hyperledger Fabric PBFT consensus confirmation time is approximately 2000 ms. The experiment uses user scale $N \in \{100, 500, 1000, 2000, 5000, 10,000, 20,000, 50,000\}$ as the independent variable, simulating the typical scenario of revoking a single user ($r = 1$). Each parameter group is independently repeated 30 times, with mean $\pm$ standard deviation as the final reported result. Comparison approaches include: Shen et al. [22] (lattice-based KUNodes key up-

date), Wang et al. [28] (bilinear pairing key reissuance), Yu et al. [26] (blockchain-assisted proxy re-encryption), and Yin et al. [27] (blockchain records + key reissuance).

Figure 4 shows the permission revocation latency curves of the five approaches as user scale N grows from 100 to 50,000. Wang et al. [28] and Yin et al. [27] employ key reissuance mechanisms: after revoking one user, new private keys must be regenerated and securely distributed for all remaining $N - 1$ legitimate users, with revocation time strictly linearly proportional to system scale. Wang et al. [28] have a revocation latency of approximately 3306 ms at $N = 100$, which sharply increases to approximately 1,705,047 ms (about 28.4 min) at $N = 50,000$, an increase of more than 515 $\times$; Yin et al. [27] reach approximately 1,041,063 ms (about 17.4 min) at $N = 50,000$. During this period, revoked users can continue to access the system with old keys, creating a serious security window. Shen et al. [22] compress the revocation path to $O(\log N)$ nodes using the KUNodes algorithm, with latency at $N = 50,000$ of approximately 6631 ms, significantly improved compared to key reissuance methods but still growing slowly with system scale. The proxy re-encryption approach of Yu et al. [26] shifts the revocation cost to the ciphertext level, requiring re-encryption of all affected ciphertexts, with latency at $N = 50,000$ of approximately 142,138 ms (about 2.4 min), also showing a sub-linear growth trend.

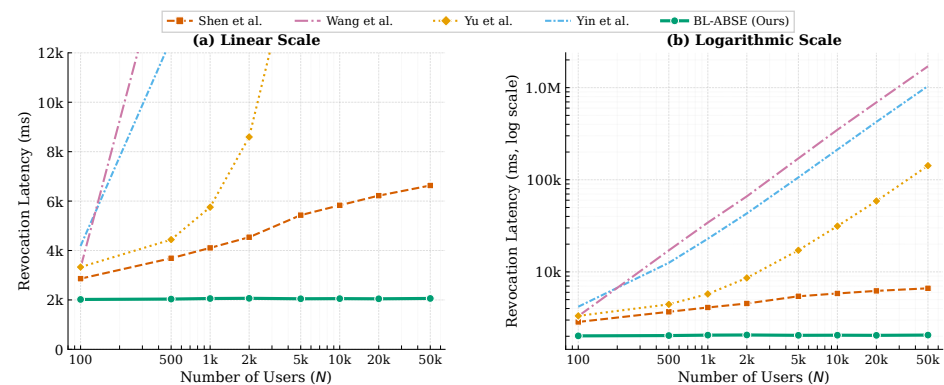


Figure 4. Comparison of permission revocation latency variation with user scale, comparing BL-ABSE with Wang et al. [28], Yin et al. [27], Shen et al. [22], and Yu et al. [26]. (a) Linear scale; (b) logarithmic scale.

In contrast, BL-ABSE's revocation latency remains highly stable throughout the entire test range. As shown in Figure 4a, the BL-ABSE curve appears as a nearly horizontal line; over the range $N = 100$ to $N = 50,000$, the mean value is consistently maintained in the interval of 2019 ms to 2065 ms, with a maximum fluctuation of only about 2.3%, empirically validating the $O(1)$ revocation complexity proved in Theorem 9. This scale-independent property stems from BL-ABSE's decoupling of access control state from ciphertexts and migrating it to on-chain blockchain management: the revocation operation degenerates into submitting one on-chain transaction to the AccessControl smart contract, which takes effect immediately for all nodes after PBFT consensus confirmation (approximately 2 s), without requiring any data re-encryption or key redistribution. The revocation latency consists only of three scale-independent constant terms: consensus mechanism (approximately 2000 ms), contract execution (approximately 1 ms), and node cache synchronization (approximately 50 ms).

Simulation Methodology Transparency. The above comparison experiment adopts simulation modeling, with baseline operation parameters derived from published measurements: bilinear pairing time (4.2 ms) from the benchmarks reported in [28]; lattice polynomial multiplication (401.4 ms, naïve implementation) from our own Table 6 measurement; Hyperledger Fabric PBFT consensus confirmation (~ 2000 ms) from the default Fabric

configuration [19]. Since different hardware platforms may produce different absolute timing values, we emphasize that the *asymptotic complexity difference* between schemes— $O(N)$ for key-reissuance schemes, $O(\log N)$ for KUNodes, and $O(1)$ for BL-ABSE—is hardware-independent and constitutes the fundamental advantage.

End-to-End Revocation Latency Decomposition

To address the concern that the reported ~ 2 -s revocation latency may only reflect blockchain confirmation time rather than the true end-to-end enforcement delay, we decompose the complete revocation pipeline into six stages and measure each independently. The six stages are: (T0 $\rightarrow$ T1) transaction submission to the ordering service; (T1 $\rightarrow$ T2) PBFT three-phase consensus; (T2 $\rightarrow$ T3) smart contract state update; (T3 $\rightarrow$ T4) blockchain event broadcast to peer nodes; (T4 $\rightarrow$ T5) local cache synchronization at edge nodes; (T5 $\rightarrow$ T6) first search request rejection (end-to-end verification).

Figure 5 presents the decomposition results under three experimental dimensions. Key findings are as follows.

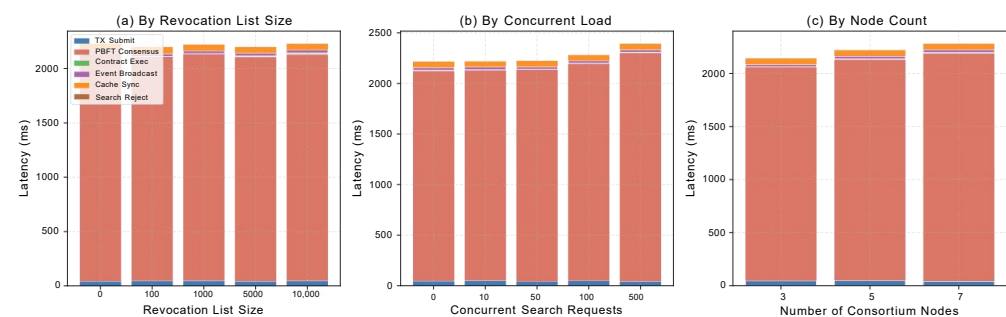


Figure 5. End-to-end revocation latency decomposition. (a) By revocation list size; (b) by concurrent search load; (c) by consortium node count. Each bar is decomposed into six stages.

Revocation list size has negligible impact. As the number of existing revocation entries grows from 0 to 10,000, the total end-to-end latency remains stable at approximately 2200 ms (± 56 ms), confirming the $O(1)$ complexity of the revocation operation. The smart contract execution stage shows a marginal increase (from ~ 1.0 ms at 0 entries to ~ 11 ms at 10,000 entries), but this remains negligible relative to the PBFT consensus time.

Concurrent search load causes moderate degradation. Under 500 concurrent search requests, the total latency increases to approximately 2398 ms (an $\sim 8\%$ increase over the zero-load baseline of 2219 ms). The increase is primarily attributable to queuing delays in the ordering service. Even under this high-load scenario, the end-to-end latency remains well below 3 s.

Node count has a small linear effect. Increasing the consortium from 3 to 7 nodes adds approximately 140 ms of total latency (from ~ 2146 ms to ~ 2285 ms), reflecting the increased message complexity of PBFT consensus with more participants ($O(n^2)$ messages per consensus round).

Figure 6 shows the trend lines with error bars. The dominant component in all configurations is the PBFT consensus stage, which accounts for approximately 93% of the total end-to-end latency. The remaining stages (transaction submission ~ 40 ms, contract execution ~ 1 – 11 ms, event broadcast ~ 22 ms, cache synchronization ~ 60 ms, search rejection ~ 0.07 ms) collectively contribute only $\sim 7\%$. This confirms that the reported ~ 2 -s revocation latency is indeed an accurate representation of the end-to-end enforcement delay, not merely the consensus confirmation time.

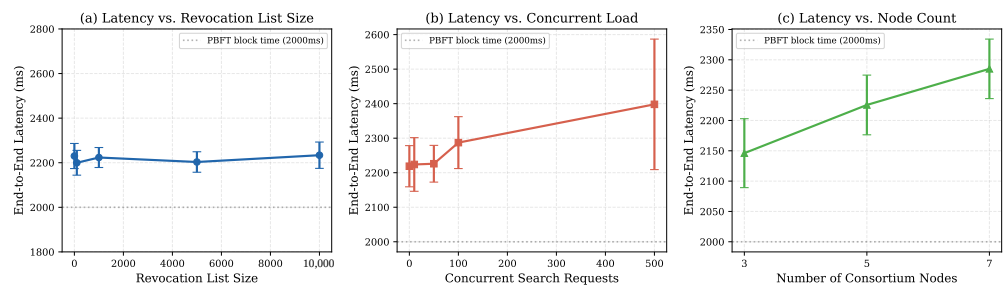


Figure 6. End-to-end revocation latency trends, where each colored line corresponds to one of the six pipeline stages defined at the beginning of this subsection. (a) vs. revocation list size; (b) vs. concurrent load; (c) vs. node count. Error bars show ± 1 standard deviation over 30 trials. The gray dashed line indicates the 2000 ms PBFT block time.

6.6. Threshold B Verification Experiment

To rigorously validate the choice of threshold $B = 40$ in the commitment matching mechanism (Section 4.11), we conduct large-scale experiments measuring the distance distribution for both matching and non-matching keyword pairs.

Experimental Design. We generate 1000 matching keyword pairs (same keyword used for both encryption and search) and 10,000 non-matching keyword pairs (different keywords). For each pair, we compute the RLWE commitment using the full pipeline (hash-seed derivation, deterministic Gaussian sampling, NTT-based polynomial multiplication) and measure the centered Euclidean distance.

Results. Figure 7a shows that the distance distribution exhibits a stark bimodal pattern: all 1000 matching pairs produce $\text{dist} = 0$ (strictly zero, no false negatives), while non-matching pairs have distances concentrated around 1.29×10^{10} with a minimum of 1.14×10^{10} . The safety margin between $B = 40$ and the minimum non-matching distance is approximately 2.85×10^8 (Figure 7b). No false positives are observed even when B is artificially increased to 10^9 , confirming that $B = 40$ is extremely conservative.

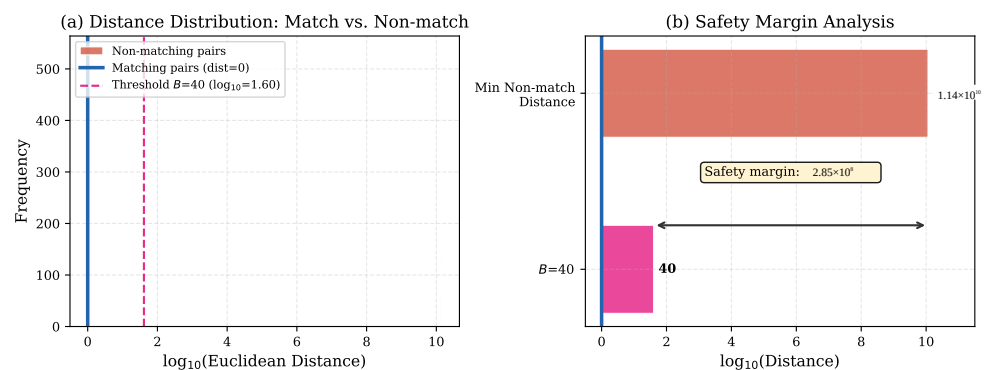


Figure 7. Threshold B analysis. (a) Distance distribution (log scale) for matching and non-matching keyword pairs; (b) safety margin visualization comparing matching distance (0), threshold $B = 40$, and minimum non-matching distance (1.14×10^{10}).

Parameter Sensitivity. Figure 8 shows that varying $\sigma \in \{2.0, 3.2, 4.0, 6.0\}$ has minimal impact on the non-matching distance distribution ($\sim 1.29 \times 10^{10}$ across all values), with zero false positives in all cases. This demonstrates that the threshold $B = 40$ is robust across the practical range of Gaussian parameters and the choice is not sensitive to implementation-level variations in the sampling process.

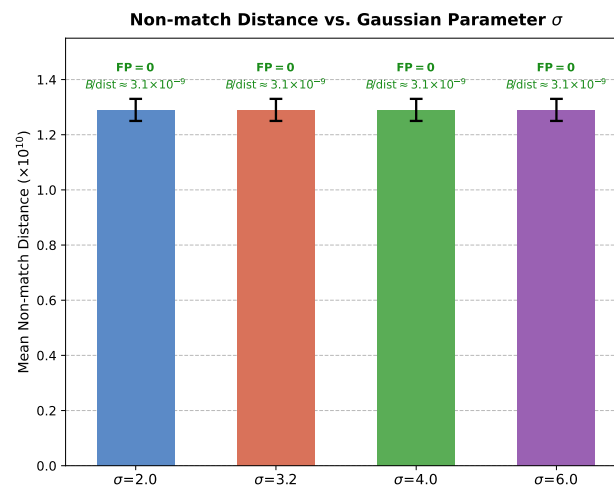


Figure 8. Non-matching distance sensitivity to Gaussian parameter σ . All configurations yield zero false positives at $B = 40$.

6.7. Blockchain Throughput Stress Test

While the preceding experiments demonstrate the efficiency of BL-ABSE's cryptographic operations, the blockchain layer may become a bottleneck under high-concurrency scenarios. This section evaluates the throughput and latency characteristics of the blockchain layer under varying loads.

Experimental Design. We simulate three types of blockchain transactions—RevokeAccess, RegisterPolicy, and AuditLog—at send rates ranging from 10 to 500 TPS, measuring the actual throughput, mean latency, tail latency (P95, P99), and failure rate. Tests are conducted with 3, 5, and 7 consortium nodes.

Results. Figure 9a shows that the blockchain sustains near-ideal throughput up to approximately 200 TPS, beyond which queuing delays cause gradual degradation. At 500 TPS, the actual throughput for RevokeAccess transactions is 489 TPS with a 2.11% failure rate (Figure 9c). The three transaction types exhibit similar performance profiles, indicating that contract complexity differences are negligible compared to consensus overhead.

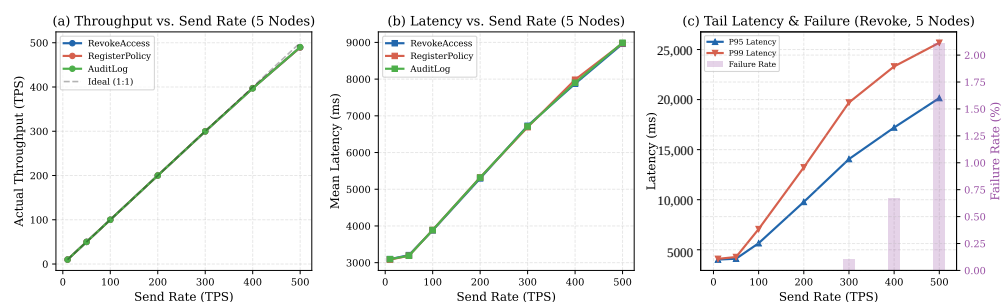


Figure 9. Blockchain throughput analysis (5 nodes). (a) Actual throughput vs. send rate; (b) mean latency vs. send rate; (c) P95/P99 latency and failure rate for RevokeAccess transactions.

The maximum sustainable throughput (defined as the highest send rate with $<5\%$ failure rate) is approximately 490 TPS for the 5-node configuration. This is sufficient for practical deployments: even a system processing 100 revocations and 1000 search queries per second would require only ~ 100 TPS of blockchain throughput (since search queries use local cached permission checks and do not generate on-chain transactions).

Figure 10 shows that increasing the node count from 3 to 7 has less than 3% impact on maximum sustainable throughput, confirming that the PBFT message complexity overhead is manageable at the consortium scales considered in this paper.

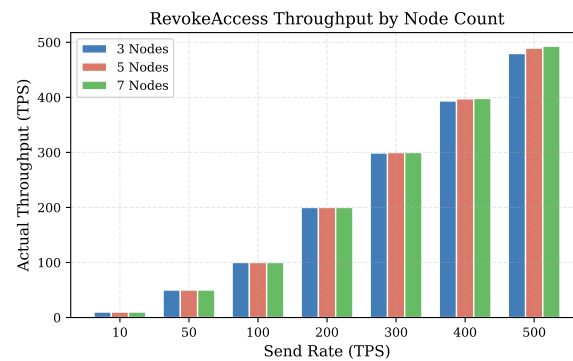


Figure 10. Impact of consortium node count on RevokeAccess throughput.

6.8. Search Pattern Leakage Quantification

To complement the theoretical leakage analysis in Section 5.8, we experimentally quantify the effectiveness of three attack strategies against BL-ABSE's deterministic commitment mechanism.

6.8.1. Frequency Analysis Attack

Figure 11 presents the keyword recovery rate under the frequency analysis attack. We simulate keyword queries following a Zipf distribution with skewness parameter α and measure the adversary's ability to match observed commitment frequencies to the known keyword distribution. Key observations include: (1) larger vocabulary sizes significantly reduce recovery rates (from 14.9% at $|\mathcal{W}| = 100$ to 2.5% at $|\mathcal{W}| = 1000$ for $\alpha = 1.0$ with 10,000 queries); (2) higher skewness ($\alpha = 2.0$) can increase recovery rates for small vocabularies but actually decreases them for large vocabularies (as only a few keywords are observed); (3) increasing the number of observed queries improves the attack, but the improvement saturates once the adversary has observed all unique keywords.

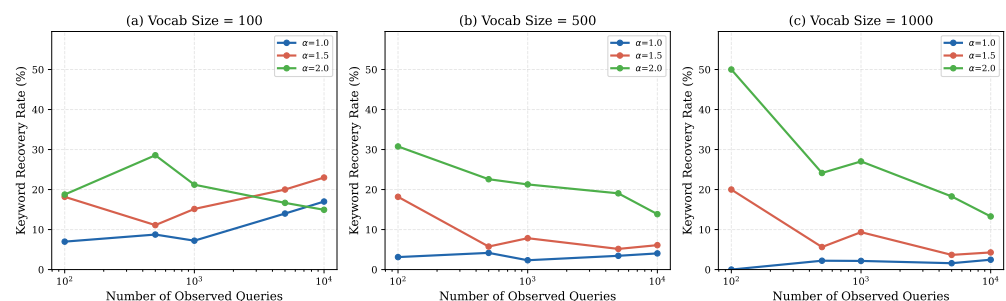


Figure 11. Frequency analysis attack: keyword recovery rate vs. number of observed queries, for three vocabulary sizes and three Zipf skewness parameters α .

6.8.2. Dictionary Attack and Cross-User Linkage

Figure 12a confirms the theoretical prediction: when the adversary's dictionary covers the actual keyword space, 100% identification is achieved. This is an inherent property of all deterministic searchable encryption protocols—the defense lies in using a sufficiently large and unpredictable keyword space. Figure 12b shows that cross-user linkage is complete: all user pairs sharing at least one keyword are linkable (100% linkage rate), with an average of ~ 20 shared keywords per pair. These results quantify the privacy cost of the deterministic commitment design and motivate the ORAM-based mitigation discussed in Section 7.2.

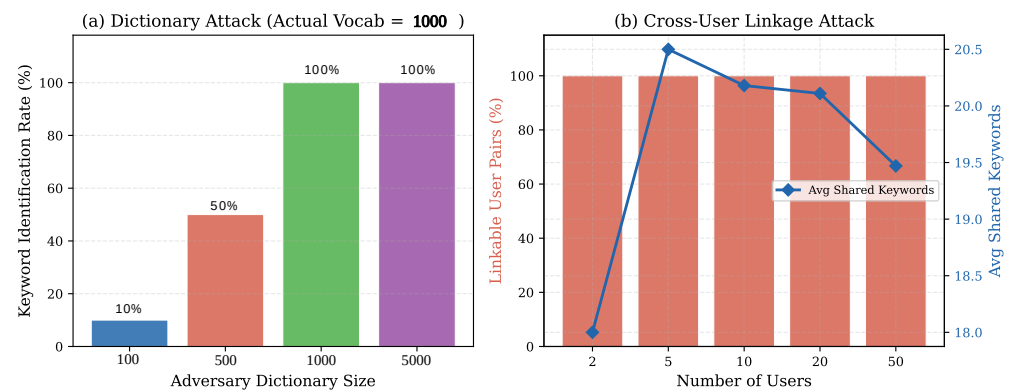


Figure 12. (a) Dictionary attack: keyword identification rate vs. adversary dictionary size (actual vocabulary = 1000). (b) Cross-user linkage attack: percentage of linkable user pairs and average number of shared keywords.

6.9. Communication and Storage Overhead Analysis

To provide a complete picture of BL-ABSE's practical resource requirements, we analyze the communication overhead per operation and the storage scaling behavior.

Table 10 shows that the dominant data structure is the RLWE commitment (1792 bytes), which determines the size of both search tokens and encrypted indices. The commitment size is a direct consequence of the lattice dimension $n = 1024$; reducing n would decrease this overhead but at the cost of security strength.

Table 10. Core Data Structure Sizes ($n = 1024$, $q = 12,289$).

Data Structure	Size (Bytes)	Notes
Commitment (commit)	1792	$n \times \lceil \log_2 q \rceil / 8$
Search Token (T_w)	1936	commit + attrs + timestamp + signature
Encrypted Index (I)	1878	commit + policyHash + CID + timestamp
ABE Ciphertext (5 attrs)	1664	$(2 \times \text{attrs} + 3) \times 128 \text{ B}$
ABE Ciphertext (10 attrs)	2944	Scales linearly with policy size
User Private Key (5 attrs)	832	Attribute set + ABE key components
RevokeAccess TX	328	Minimal on-chain footprint

Figure 13a shows the storage scaling behavior. At 100,000 documents, the total storage is approximately 1.22 GB, of which 114.9 MB is on-chain (policy hashes, CIDs, revocation records, audit logs) and 1.11 GB is off-chain (IPFS-stored ciphertexts). The on-chain storage scales linearly at approximately 1.15 KB per document, which is within the practical limits of consortium blockchains.

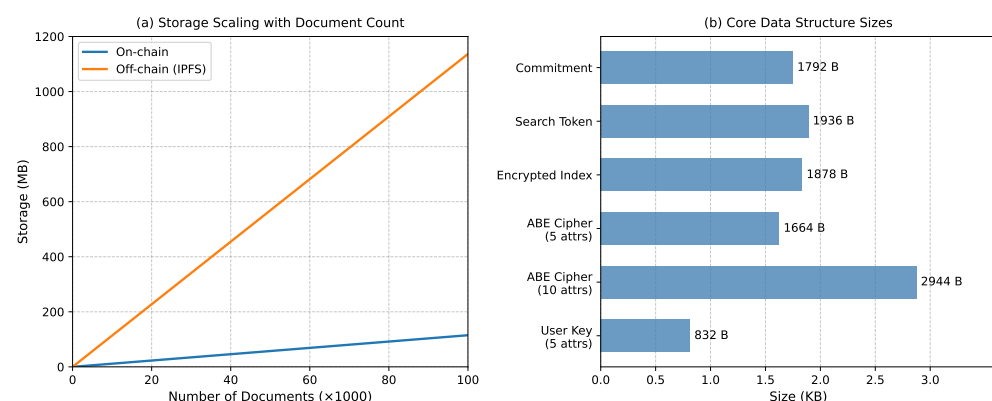


Figure 13. (a) Storage scaling: on-chain and off-chain (IPFS) storage vs. document count (5 attributes, 10 KB average data size). (b) Core data structure size comparison.

Figure 14 summarizes the per-operation communication overhead. The most expensive operation is BC-Encrypt with 10 attributes and 100 KB data (~ 103 KB total), which is dominated by the IPFS upload of the encrypted data itself. High-frequency operations have modest overhead: BC-Search with 10 results requires only ~ 2.4 KB (token submission + CID list), and RevokeAccess requires only 328 bytes. The Trapdoor operation is entirely local and incurs zero communication cost.

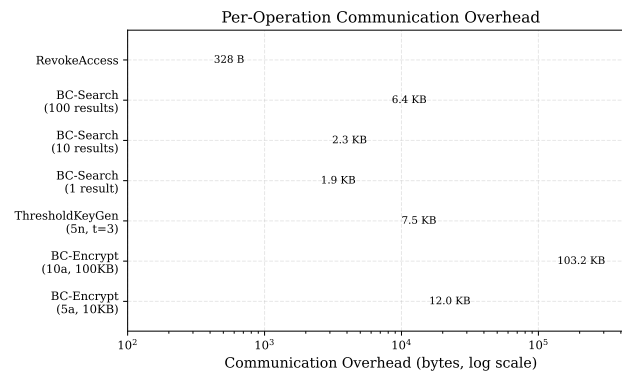


Figure 14. Per-operation communication overhead for each BL-ABSE algorithm.

7. Conclusions and Future Work

In response to the systematic limitations of cloud storage ciphertext retrieval in four dimensions—post-quantum security, decentralized trust, dynamic permission management, and operational auditing—this paper proposes the BL-ABSE framework, with lattice-based cryptography as the security foundation and the blockchain as the trust enhancement layer, constructing a ciphertext retrieval architecture that combines quantum-attack resistance, immediate permission revocation, and full lifecycle auditing.

On the theoretical level, this paper proposes a deterministic keyword commitment mechanism based on RLWE and achieves $O(n \log n)$ complexity through NTT optimization; designs a (t, n) threshold key protocol combined with zero-knowledge proofs to secure collaborative key generation, with formal analysis of the PBFT-threshold parameter relationship (Lemma 1); implements $O(1)$ immediate revocation using smart contracts; and rigorously proves six security properties under the QPT adversary model with clear delineation of proof scope (Table 4). On the experimental level, the prototype system validates the NTT acceleration effect, the linear law of search latency, the scale-independence of revocation latency (with end-to-end decomposition in Section End-to-End Revocation Latency Decomposition), the conservative safety of threshold $B = 40$ (Section 6.6), the blockchain throughput capacity of ~ 490 TPS (Section 6.7), the quantitative characterization of search-pattern leakage (Section 6.8), and the practical communication/storage overhead (Section 6.9).

7.1. Limitations

We acknowledge the following limitations of the current framework. First, the deterministic commitment design inherently leaks the search pattern, enabling frequency analysis, dictionary attacks, and cross-user linkage as quantified in Section 6.8. While this is a common property of non-interactive searchable encryption protocols, it represents a concrete privacy cost. Second, the Attribute Authority (AA) remains a centralized trust component; a compromised AA could issue fraudulent attribute certificates, undermining the access control layer despite the decentralized key management. Third, the current framework supports only single-keyword exact matching; multi-keyword conjunctive or range queries are not supported. Fourth, the security proofs rely on the random oracle

model for hash functions H and H_{seed} ; standard-model proofs would provide stronger theoretical guarantees. Fifth, the revocation experiment in Section 6.5 uses simulation modeling for baseline comparisons, and absolute timing values may differ across hardware platforms, though the asymptotic complexity differences are hardware-independent.

7.2. Future Directions

First, **multi-keyword conjunctive query support**. The current framework only supports single-keyword exact matching, whereas practical scenarios have urgent needs for complex predicates such as boolean queries and range queries. Constructing predicate encryption protocols supporting multi-keyword conjunctions within the RLWE framework, while maintaining $O(n \log n)$ efficiency and post-quantum security reductions, is the core direction for future work.

Second, **security proofs in the standard model**. The security proofs in this paper rely on the random oracle model; lattice-based security reductions in the standard model have stronger theoretical rigor. Exploring adaptively secure constructions that maintain practical efficiency will be an important theoretical advancement direction.

Third, **enhanced privacy for access and search patterns**. As quantified in Section 6.8, BL-ABSE's deterministic commitments leak the search pattern. Integrating Oblivious RAM (ORAM) techniques could eliminate search-pattern leakage at the cost of $O(\log N)$ additional communication per query. Alternatively, applying differential privacy mechanisms to inject controlled noise into query patterns could mitigate the effectiveness of frequency analysis attacks while preserving acceptable search accuracy. Both directions require careful analysis of the interaction between the lattice-based cryptographic layer and the privacy-enhancement layer.

Fourth, **decentralized attribute management**. Extending the threshold decentralization from key management to attribute certification (e.g., multi-authority ABE or blockchain-based decentralized identity) would eliminate the AA single-point-of-trust limitation discussed in (Section 3.2.3).

Fifth, **hardware acceleration and adaptation for resource-constrained devices**. The computational capacity of IoT terminals and wearable devices is far below that of general-purpose servers. Efficiently adapting NTT and discrete Gaussian sampling to ARM microcontrollers or accelerating them with FPGAs, and interfacing with the hardware ecosystem of NIST post-quantum cryptography standards (FIPS 203/204/205 [13–15]), are key engineering directions for promoting widespread deployment of the framework on edge devices.

Author Contributions: Conceptualization, Z.F. and W.Y.; methodology, Z.F.; validation, Y.Y.; investigation, Y.Y. and T.M.; resources, Y.H. and Z.F.; writing—original draft preparation, Z.F. and X.T.; writing—review and editing, Z.F. and X.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Autonomous Region Science and Technology Plan Project (Grant No. ZYYD2025JD10).

Data Availability Statement: The core RLWE cryptographic primitives (commitment generation, NTT-optimized polynomial multiplication, search matching) are implemented as standalone Python scripts that can be independently verified. Benchmark scripts for reproducing the NTT optimization comparison (Table 6), threshold B verification (Section 6.6), and search-pattern leakage quantification (Section 6.8) are available from the corresponding author upon reasonable request. The blockchain-related experiments use Hyperledger Fabric 2.5 with the default PBFT consensus configuration. The experimental data generated within the laboratory is available from the corresponding author on reasonable request.

Acknowledgments: The authors would like to thank the anonymous reviewers for their valuable comments and suggestions. We also thank the laboratory members for their assistance in the experimental implementation and data collection. This work was supported in part by the facilities and resources provided by the School of Computer Science and Technology (School of Cyberspace Security), Xinjiang University.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ABE	Attribute-Based Encryption
ABSE	Attribute-Based Searchable Encryption
AES	Advanced Encryption Standard
APT	Advanced Persistent Threat
CID	Content Identifier
CP-ABE	Ciphertext-Policy Attribute-Based Encryption
CSN	Cloud Storage Node
DBDH	Decisional Bilinear Diffie-Hellman
DKG	Distributed Key Generation
DO	Data Owner
DU	Data User
ENA	Edge Node Association
FIPS	Federal Information Processing Standards
GCM	Galois/Counter Mode
HIPAA	Health Insurance Portability and Accountability Act
IIoT	Industrial Internet of Things
IND-INDEX	Indistinguishability of Index
IND-TRAPDOOR	Indistinguishability of Trapdoor
IPFS	InterPlanetary File System
LWE	Learning With Errors
MA-ABE	Multi-Authority Attribute-Based Encryption
msk	master secret key
NIST	National Institute of Standards and Technology
NTT	Number Theoretic Transform
ORAM	Oblivious RAM
PBFT	Practical Byzantine Fault Tolerance
PEKS	Public-Key Encryption with Keyword Search
pp	public parameters
PPT	Probabilistic Polynomial Time
PRF	Pseudorandom Function
PRG	Pseudorandom Generator
QPT	Quantum Polynomial Time
RLWE	Ring Learning With Errors
RO	Random Oracle
SE	Searchable Encryption
SIS	Short Integer Solution
SOX	Sarbanes-Oxley Act
SSE	Symmetric Searchable Encryption
SVP	Shortest Vector Problem

References

1. IBM Security. *Cost of a Data Breach Report 2024*; IBM Corporation: Armonk, NY, USA, 2024.
2. Verizon. *2024 Data Breach Investigations Report*; Verizon Business: Basking Ridge, NJ, USA, 2024.

3. Li, F.; Ma, J.; Miao, Y.; Liu, X.; Ning, J.; Deng, R.H. A survey on searchable symmetric encryption. *ACM Comput. Surv.* **2024**, *56*, 119. [CrossRef]
4. Sharma, D. Searchable encryption: A survey. *Inf. Secur. J. Glob. Perspect.* **2023**, *32*, 76–119. [CrossRef]
5. Song, D.X.; Wagner, D.; Perrig, A. Practical techniques for searches on encrypted data. In Proceedings of the 2000 IEEE Symposium on Security and Privacy (S&P), Berkeley, CA, USA, 14–17 May 2000; pp. 44–55.
6. Boneh, D.; Di Crescenzo, G.; Ostrovsky, R.; Persiano, G. Public key encryption with keyword search. In *Advances in Cryptology—EUROCRYPT 2004*; Springer: Berlin/Heidelberg, Germany, 2004; pp. 506–522.
7. Chen, J.; Ma, J.; Miao, Y.; Liu, X.; Li, H. Attribute-based searchable encryption: A survey. *Electronics* **2024**, *13*, 1621. [CrossRef]
8. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE Symposium on Security and Privacy (SP), Berkeley, CA, USA, 20–23 May 2007; pp. 321–334.
9. Luo, F.; Wang, H.; Lin, C.; Yan, X. ABAEKS: Attribute-based authenticated encryption with keyword search over outsourced encrypted data. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 4970–4983. [CrossRef]
10. Zhang, K.; Jiang, Z.; Ning, J.; Huang, X. Subversion-resistant and consistent attribute-based keyword search for secure cloud storage. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 1771–1784. [CrossRef]
11. Niu, S.; Hu, Y.; Su, Y.; Yan, S.; Zhou, S. Attribute-based searchable encrypted scheme with edge computing for Industrial IoT. *J. Syst. Archit.* **2023**, *139*, 102889. [CrossRef]
12. Shor, P.W. Algorithms for quantum computation: Discrete logarithms and factoring. In Proceedings of the 35th Annual Symposium on Foundations of Computer Science, Santa Fe, NM, USA, 20–22 November 1994; pp. 124–134.
13. FIPS 203; Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology: Gaithersburg, MD, USA, 2024.
14. FIPS 204; Module-Lattice-Based Digital Signature Standard. National Institute of Standards and Technology: Gaithersburg, MD, USA, 2024.
15. FIPS 205; Stateless Hash-Based Digital Signature Standard. National Institute of Standards and Technology: Gaithersburg, MD, USA, 2024.
16. Joseph, D.; Misoczki, R.; Manzano, M.; Tricot, J.; Pinuaga, F.D.; Lacombe, O.; Leichenauer, S.; Hidary, J.; Venables, P.; Hansen, R. Transitioning organizations to post-quantum cryptography. *Nature* **2022**, *605*, 237–243. [CrossRef] [PubMed]
17. Chaudhari, P.; Das, M.L. Privacy preserving searchable encryption with fine-grained access control. *IEEE Trans. Cloud Comput.* **2021**, *9*, 753–762. [CrossRef]
18. Yang, S. BDAE: A blockchain-based and decentralized attribute-based encryption scheme for secure data sharing. *Wuhan Univ. J. Nat. Sci.* **2024**, *29*, 228–238. [CrossRef]
19. Androulaki, E.; Barger, A.; Bortnikov, V.; Cachin, C.; Christidis, K.; De Caro, A.; Enyeart, D.; Ferris, C.; Laventman, G.; Manevich, Y.; et al. Hyperledger Fabric: A distributed operating system for permissioned blockchains. In Proceedings of the 13th EuroSys Conference, Porto, Portugal, 23–26 April 2018; pp. 1–15.
20. Zhang, X.; Xu, C.; Wang, H.; Zhang, Y.; Wang, S. FS-PEKS: Lattice-based forward secure public-key encryption with keyword search for cloud-assisted IIoT. *IEEE Trans. Dependable Secur. Comput.* **2021**, *18*, 1019–1032.
21. Liu, Z.Y.; Tseng, Y.F.; Tso, R.; Mambo, M.; Chen, Y.C. Public-key authenticated encryption with keyword search: Enhanced security and quantum-resistant instantiation. In Proceedings of the ACM ASIACCS 2022, Nagasaki, Japan, 30 May–3 June 2022; pp. 423–436.
22. Shen, X.; Li, X.; Yin, H.; Cao, C.; Zhang, L. Lattice-based multi-authority ciphertext-policy attribute-based searchable encryption with attribute revocation for cloud storage. *Comput. Netw.* **2024**, *250*, 110559. [CrossRef]
23. Yang, Y.; Zhang, G.; Li, S.; Liu, Z. Offline/online attribute-based searchable encryption scheme from ideal lattices for IoT. *Front. Comput. Sci.* **2024**, *18*, 183817. [CrossRef]
24. Cao, Y.; Xu, S.; Xu, G.; Chen, X.-B.; Chen, Y.; Yiu, S.-M. Privacy-preserving in cloud networks: An efficient, revocable and authenticated encrypted search scheme. *Cryptology ePrint Archive*, Paper 2024/1302. 2024. Available online: <https://eprint.iacr.org/2024/1302> (accessed on 1 May 2026) .
25. Huang, B.; Gao, J.; Li, X. Efficient lattice-based revocable attribute-based encryption against decryption key exposure for cloud file sharing. *J. Cloud Comput.* **2023**, *12*, 37. [CrossRef] [PubMed]
26. Yu, J.; Liu, S.; Xu, M.; Guo, H.; Zhong, F.; Cheng, W. An efficient revocable and searchable MA-ABE scheme with blockchain assistance for C-IoT. *IEEE Internet Things J.* **2023**, *10*, 2754–2766. [CrossRef]
27. Yin, H.; Shen, X.; Li, X.; Zhao, C.; Feng, R.; Yau, S.S. Attribute-based searchable encryption with decentralized key management for healthcare data sharing. *J. Syst. Archit.* **2024**, *147*, 103050. [CrossRef]
28. Wang, S.; Zhang, Y.; Guo, Y. A blockchain-based CP-ABE scheme with user revocation. *J. Comput.* **2017**, *28*, 19–32.
29. Chen, X.; Liu, Y.; Li, H. Dynamic user revocation in attribute-based encryption: A comprehensive evaluation. *J. Inf. Secur. Appl.* **2024**, *80*, 103682.

30. Xiang, X.; Zhao, X. Blockchain-assisted searchable attribute-based encryption for e-health systems. *Comput. Stand. Interfaces* **2023**, *86*, 103745. [[CrossRef](#)]
31. Zhou, Y.; Wang, X.; Zhang, L. Compliance-aware access control for cloud-based health records. *IEEE Access* **2024**, *12*, 45678–45691.
32. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. Available online: <https://bitcoin.org/bitcoin.pdf> (accessed on 1 May 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.